



Buffer sizing in critical chain project management by network decomposition[☆]

Bingling She^a, Bo Chen^{a,*}, Nicholas G. Hall^b

^a Warwick Business School, University of Warwick, Coventry CV4 7AL, UK

^b Fisher College of Business, The Ohio State University, Columbus, OH 43210, USA



ARTICLE INFO

Article history:

Received 5 August 2020

Accepted 2 December 2020

Available online 4 December 2020

Keywords:

Project management

Critical chain

Buffer sizing

Network decomposition

Simulation study

ABSTRACT

Project management organizes about 30% of the world's economy. Many recent projects apply critical chain project management (CCPM) methodology, which requires the design of project and feeding buffers. Accurate sizing of these buffers is essential, because too small buffers result in emergency procedures to prevent late delivery, whereas too large buffers result in uncompetitive bids and lost contracts. Previous buffer sizing research, focused predominantly on the critical chain, typically results in excessive buffers, and in critical chains being challenged by feeding buffers during planning. This work also performs inconsistently, for example in makespan estimation, at execution. We propose a new procedure for buffer sizing based on network decomposition, which offers logical advantages over previous ones. First, the size of a feeding buffer is determined from all associated noncritical chains. Second, the project buffer incorporates safety margins outside the critical chain by comparing feeding chains with their parallel critical counterparts. Computational testing on a case study of a real project and extensive simulated data shows that our procedure delivers much greater accuracy in estimating project makespan, and smaller feeding buffers. Furthermore, the resulting critical chain is never challenged. Additional benefits include delayed expenditure, and reductions in work-in-process, rework, and multitasking.

© 2020 Elsevier Ltd. All rights reserved.

1. Introduction

Approximately 30% of global economic activity is organized using project management, implying an annual value of about \$27 trillion [16,52]. Applications have expanded from construction and engineering, to information technology, R&D, software development, and new product and service development [12]. However, due to a lack of effective project planning or execution, many projects are delivered late or even cancelled before completion. The resulting costs are billions of dollars per year [22]. A widely used response is critical chain project management, or CCPM¹ [11]. The development of CCPM is viewed by some experts as an important breakthrough in the history of project management [13]. The planning and execution principles of CCPM have resulted in significant improvement in the performance of numerous projects. Nonetheless, a central unsolved problem within CCPM is the sizing of buffers, which is the focus of our work.

The buffer sizing problem arises as follows. In CCPM we develop a deterministic baseline schedule, i.e., identifying the critical chain as the set of tasks that determines the project makespan considering both precedence and resource dependencies, which is then protected by three types of buffers from project uncertainties and resource bottlenecks [32,38]. A single project buffer is placed at the end of the critical chain to protect the completion time of the project. Multiple feeding buffers are added to the end of noncritical chains to prevent them from becoming critical. Multiple resource buffers, which do not consume time, act as alerts for resource availability on the critical chain. The project and feeding buffers are time buffers which need to be carefully sized since they may significantly impact the planned project schedule. The original work on CCPM [11] does not include a statistically rigorous or empirically robust formula for sizing these time buffers. The subsequent literature introduces more complicated formulas, but performance remains inconsistent. We propose a new approach based on decomposition of the project network. This enables us to codetermine the feeding buffers and the project buffer more accurately. A real company buffer sizing problem is introduced in Example 1 and Appendix D.

Accurate buffer sizing is essential to the economic success of project companies. Buffers that are too small result in replanning

[☆] Area: Production Management, Scheduling and Logistics. This manuscript was processed by Associate Editor Singh.

* Corresponding author.

E-mail address: b.chen@warwick.ac.uk (B. Chen).

¹ A summary of acronyms is provided in Appendix A

and expensive emergency procedures to avoid late delivery of the project. However, buffers that are too large result in uncompetitive bidding for projects and loss of potentially valuable contracts [12]. Despite the availability of numerous methods proposed in the literature, issues remain. Previous buffer sizing research, focused predominantly on the critical chain, typically results in excessive buffers and the problems discussed above. Further, this research lacks detailed analysis of the interactions between critical and non-critical chains and between project and feeding buffers. Hence, the critical chain tends to be challenged by the insertion of feeding buffers during planning, which creates confusion, rescheduling, and the use of expensive emergency procedures. Additionally, CCPM lacks a well-defined procedure for complex networks where a given task may have both predecessors and successors from several chains [33].

Our work is apparently the first to approach buffer sizing through analytical decomposition of the project network, which enables detailed comparisons between critical and noncritical parts of the network, and guarantees the accuracy and mutual compatibility of the project and feeding buffers. Our work makes the following contributions.

- We decompose the project network to analyze the interactions of project network structure and buffers, and provide a well-defined buffer sizing procedure that is implementable for any project network whose structure is known *ex ante*.
- We resolve the problem of a challenged critical chain, while simultaneously addressing situations with multiple critical chains.
- Our procedure offers logical and computational advantages over previous ones. The size of a feeding buffer is determined based on uncertainties of *all* noncritical chains feeding it, instead of only the longest chain. The project buffer incorporates both safety margins of critical tasks and remaining safety margins of feeding chains via comparison of feeding chains with their parallel critical counterparts.
- Based on extensive computational testing for both real and simulated data, our procedure delivers project schedules with more accurate estimation of project makespans and smaller feeding buffers than benchmark methods. Additional benefits of our methodology include delayed expenditure, and reduced work-in-process, rework, and multitasking.

The remainder of this paper is organized as follows. Section 2 provides a literature review and identifies ways in which CCPM methodology is in need of improvement. Section 3 provides a formal description of the problem and our notation. Section 4 presents our buffer sizing procedure. Section 5 details our experimental design for testing the proposed procedure against several benchmark procedures from the literature. Section 6 provides the results of a computational study, using a case study of a real project and a large simulated data set, and identifies various insights of relevance to management. Section 7 presents some conclusions and suggestions for future work.

2. Literature review

Goldratt [11] describes task estimates as containing two parts: an average part and a safety margin. He removes safety margins from task durations and includes half of the sum of all safety margins along a chain in a buffer at the end. This is the original buffer sizing approach, known as Cut-and-Paste Method (C&PM). A widely used alternative method is Root Square Error Method (RSEM), where the buffer size is the square root of the sum of the squared safety margins of tasks on the associated chain [13,21,27]. Ashtiani et al. [1] assume task durations follow lognormal distributions and use a shape parameter to estimate task safety margins,

which are aggregated as in RSEM. Similarly, Fallah et al. [8] use the coefficient of variation, skewness and kurtosis of the assumed lognormal distribution to estimate buffer size. Long and Ohsato [23] employ fuzzy mathematics to estimate activity durations and safety margins. Gao et al. [9] apply grey system theory to quantify inherent task uncertainties. Their formula consists of the product of grey task duration and uncertainty proportion, where the latter depends on the grey degree and position weight of the task. Zhang et al. [47] assume that task durations follow Beta distributions and use optimistic, most likely, and pessimistic durations. They define the “elasticity” of task durations using these estimates. The buffer then incorporates the three estimates and the elasticity.

Other studies estimate buffer sizes by considering not only task safety margins, but also factors external to the individual tasks that influence project performance. These factors include network complexity, resource tightness, task flexibility, risk preference and information flow. Statistical concepts such as the normal and log-normal distributions, fuzzy mathematics and simulation techniques are used to calculate task safety margins. To quantify the external uncertainties, most of these studies use simple mathematical formulas, while several others apply fuzzy mathematics and risk assessment approaches. Leach [20] argues that buffer size should cover both task duration variances and bias, which is anything that invalidates the aggregation of variances and increases the project schedule relative to the plan. He provides a list of bias factors, including merging paths and multitasking, and suggests increasing the buffer size in proportion. Tukul et al. [38] use the variance of a lognormal distribution of task durations as the safety margin, and also incorporate two project attributes, resource tightness and network complexity, into the buffer. Shi and Gong [34] use fuzzy mathematics to quantify resource tightness and also consider the risk preferences of project managers.

Xie et al. [44] propose a modified RSEM for software development projects, by incorporating technical complexity and the tightness of resource constraints. Bie et al. [4] consider the uncertainty caused by common risk factors, such as weather. They quantify this uncertainty using the ratio of the number of risk-related tasks to the total number of tasks in a chain and the proportion of duration uncertainty of a task that is explained by the risk factor, and incorporate it into the variance-based buffer size. Ma and Jiang [25] use trapezoidal fuzzy numbers to describe task safety margins, and incorporate the influence of task positions. Yu et al. [45] propose a buffer sizing procedure by incorporating a resource utilization impact factor into the RSEM formula, and the buffers are then modified according to the free slack values of the tasks. The positive difference between the initial size of a feeding buffer and the free slack is added to the initial project buffer.

Ma et al. [26] develop a complex buffer size formula, using the variance of a normal distribution as the safety margin and five external uncertainties: environment, task or network complexity, task flexibility, resource tightness and risk preference. Zhang et al. [49] determine the buffer size by including the difference between 90% - confidence and 50% - confidence task durations as the safety margin, and the effects of resource tightness and network complexity. Further, Zhang et al. [50,51] consider comprehensive resource tightness, resource substitutability, and resource usage in variance-based buffer sizing. Zhang et al. [48] also model external uncertainty caused by information flow between tasks, and add specified rework safety time to buffers. Ghoddousi et al. [10] treat the safety margin in the same way as Ma et al. [26], but use a risk assessment approach to analyze the effects of chain complexity and chain flexibility on buffer size. Ma et al. [24] consider risks of rework, and define a rework buffer which is added to the estimated project makespan. Zarghami et al. [46] assume stochastic resource availabilities and define the resource reliability measure to scale up the RSEM buffers.

A few papers use Monte Carlo simulation to define the risk of overrunning task end time, and decide the buffer size based on this information. Hoel and Taylor [15] define the project buffer size as the difference between the simulated 90% project completion time and expected project completion time, but specify the feeding buffer size using free slack. However, Tenera [35] determines both the project and feeding buffers via Monte Carlo simulation. Other works use stochastic models to find task start times and project completion time. Trietsch [36] considers project buffer size for a completely serial project. He solves a stochastic model of the project completion time to minimize project cost including delay penalty, and determines the project buffer based on the standard deviation of the completion time and a desired service level, i.e., probability of on-time completion. A correction for task duration estimation bias is also considered. Trietsch [37] defines a feeding buffer as the nonnegative difference between the latest start time and the actual start time of a task. He derives actual start times from a stochastic program that minimizes linear earliness and tardiness costs, subject to precedence constraints.

The existing literature does not integrate the project network structure, when known *ex ante*, into decisions about the sizes of buffers. Raz et al. [33] raise this concern by pointing out that there is no clear buffer sizing procedure for complex networks where a given task has both predecessors and successors from several chains. Safety margins of such noncritical tasks could be unnecessarily covered by several feeding buffers. The literature ignores this issue and uses global characteristics such as network complexity to scale up the buffers. As a result, published procedures do not provide robust estimates of buffer sizes and project makespans for diverse data sets. Our work directly uses local network structure to address this issue.

The issue of a challenged critical chain is discussed by Hoel and Taylor [15], Herroelen and Leus [13], Herroelen et al. [14] and Raz et al. [33]. An empirical study by Zhao et al. [53] further show that a challenged critical chain occurs frequently over 110 projects. Three types of solutions to this issue exist in the literature. Tukel et al. [38] keep the critical chain unchanged even if one of the feeding chains becomes a longer path. Leach [22, p.174] uses the same idea and claims this issue is insignificant in larger projects. The second solution determines feeding buffer sizes by considering the free slacks of feeding chains [15,17,31,45]. The third solution is by Zhao et al. [53] who propose a two-stage heuristic rescheduling approach. These solutions either determine the feeding buffer without considering uncertainties of noncritical chains, or simply add the positive difference of safety margin and free slack of each feeding buffer to the project buffer. Doing so ignores parallel relationships between these buffers and results in unnecessarily large project buffers. Herroelen and Leus [13] suggest calculating the project buffer according to the baseline schedule that already includes the feeding buffers. However, they do not offer sufficient details for implementation. Our work addresses these issues by incorporating uncertainty and both critical and noncritical chain information into buffer sizing decisions.

3. Preliminaries

Section 3.1 provides a description of the buffer sizing problem. Section 3.2 describes how task safety margins, an important building block in buffer sizing, are calculated. Section 3.3 introduces the extended project network in which our buffer sizing procedure operates.

3.1. Problem description

We consider a project consisting of a task set $V = \{1, 2, \dots, n\}$, a set $E \subseteq V \times V$ of precedence relationships, and a renewable re-

source set $R = \{1, 2, \dots, m\}$. Tasks 1 and n are dummy tasks, with no duration or resource requirement, representing the start and end points of the project. Thus, task 1 is an immediate predecessor of every non-dummy task without other non-dummy predecessors, and task n is an immediate successor of every non-dummy task without other non-dummy successors. We assume there is a constant availability r_k for each resource $k \in R$ throughout the project horizon, which is a standard assumption in CCPM and resource constrained project scheduling problems [38]. Every non-dummy task i has a non-preemptive stochastic duration D_i with mean d_i and variance var_i , and a constant resource demand $r_{ik_i} \leq r_{k_i}$ of resource $k_i \in R$. Note the resource k_i is the primary resource that determines the duration of task i [22, p.170]. For each task i , let $\Gamma_i^{-1} \subseteq V \setminus \{i\}$ denote the set of its immediate predecessors. The set E of precedence relationships is given as $E = \{(i, j) : i \in \Gamma_j^{-1}, j \in V\}$. For every pair $(i, j) \in E$, task i must be finished before the start of j . All the tasks and precedence relationships form an acyclic task-on-node network $\text{PN}(V, E)$ [18]. In this network, precedence relationships (i, j) are denoted by $i \rightarrow j$ when referring to chains, and tasks are topologically numbered, meaning that if $(i, j) \in E$, then $i < j$. A *resource contention* is a situation where the total resource demand exceeds the resource availability during some time period of the project.

Fig. 1 provides an overview of the main steps of our buffer sizing procedure.

The first step in buffer sizing is critical chain identification, i.e., the identification of a baseline schedule without resource contentions, considering mean task durations d_i . The CCPM solves this problem deterministically and protects the schedule using buffers [13]. It is a special case of the classical resource constrained project scheduling problem (RCPS, Demeulemeester and Herroelen [6, p. 203]), where each task requires only one type of resource in R . This special case is strongly NP-hard, even when $m = 2$, $d_i = 1$ and $r_{ik_i} = r_{k_i} = 1$ [3]. The literature describes several heuristics to solve the classical RCPS, with the objective of minimizing makespan. However, we propose a new heuristic, described in Appendices B and C, to solve this special case RCPS. Our heuristic is designed to produce a few additional precedence relationships that break resource contentions, and also short project makespans. The output is the extended precedence relationship set that defines the extended project network for our buffer sizing procedure. From the extended network, we identify the critical chain and other noncritical chains via the critical path method. Tasks on the critical chain are *critical tasks* and others are *noncritical tasks*.

Next, a project buffer and feeding buffers need to be located at the end of chains with well-defined sizes. Our proposed buffer sizing procedure starts with the following three questions: Given a critical chain, what are the corresponding feeding chains? What are the relationships between network structures and buffers, and also among the buffers themselves? How can buffer sizes be derived from the network structure and the relationships among buffers?

We now provide an illustration of a challenged critical chain. Consider the project network in Fig. 2 of Leach [22, p. 173]. The critical chain “4 → 5 → 3 → 6” is given in the top network in Fig. 3. We insert the project buffer and feeding buffers as in the bottom network. The buffer size is decided by the Cut and Paste method [11]. With a feeding buffer added to task 2, the other chain, “2 + FB → 3 → 6”, becomes longer than the original critical chain.

3.2. Task safety margins

Technically, a task safety margin is the difference between its safe and mean durations, and allows for uncertainty in the task duration. As discussed by Patrick [30], task estimates are probabilistic



Fig. 1. The proposed buffer sizing procedure.

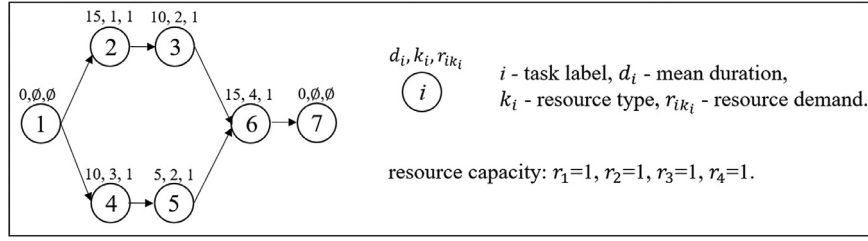


Fig. 2. A small project network.

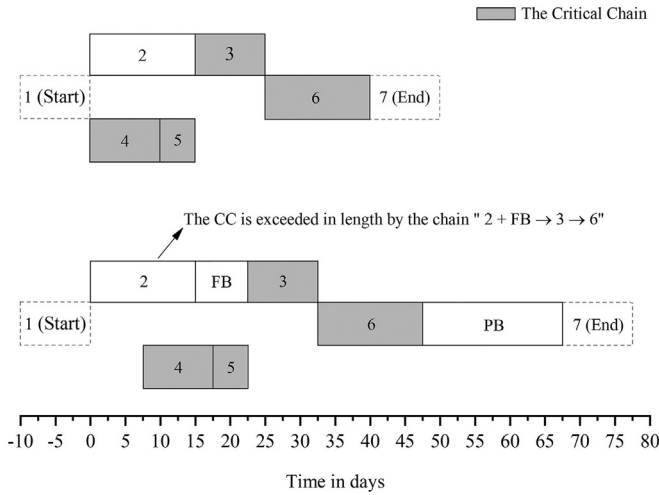


Fig. 3. A challenged critical chain.

and used under the assumption of uncertainty. Hence, we define the $p \cdot 100\%$ -level safe duration as the number that guarantees a probability p of task completion, where the value of p depends on task characteristics and risk preferences. Given a mean task duration d_i of task i , we denote its *general safety margin* by $SM(i)$ and specify its p -level *safety margin* by $SM_p(i)$. Intuitively, project and feeding buffers absorb uncertainty by incorporating the safety margins of tasks on the critical and noncritical chains, respectively.

We assume that task durations $\{D_i\}$ are independent lognormally distributed random variables with known means and standard deviations. Many works on buffer sizing, such as Tukel et al. [38], Bie et al. [4] and Zhang et al. [48], also use the lognormal distribution to describe uncertain task durations. Alternatively, the Beta and normal distributions can be used. However, the former requires specification of four parameters rather than two [39], and the latter is unsuitable since task durations are always positive. Based on this assumption, we calculate task safety margins as follows. Let the stochastic duration of task i be $D_i \sim \text{LN}(\mu_i, \sigma_i)$. Then, the mean d_i and variance var_i of D_i can be written as $d_i = \exp(\mu_i + (\sigma_i)^2/2)$ and

$$\text{var}_i = \exp(2\mu_i + (\sigma_i)^2) (\exp((\sigma_i)^2) - 1) = (d_i)^2 (\exp((\sigma_i)^2) - 1).$$

Conversely, μ_i and σ_i can be represented by d_i and var_i as $(\sigma_i)^2 = \ln(1 + \text{var}_i/(d_i)^2)$ and

$$\mu_i = \ln(d_i) - (\sigma_i)^2/2 = \ln(d_i) - \ln(1 + \text{var}_i/(d_i)^2)/2.$$

Then the p -level safe duration is $l_p(D_i) = \exp(\mu_i + z_p \cdot \sigma_i)$, where l_p and z_p are the $p \cdot 100\%$ -quantiles of the lognormal distribution and standard normal distribution [13, p. 565], respectively. Finally, we have

$$\begin{aligned} SM_p(i) &= l_p(D_i) - d_i = \exp(\ln(d_i) - (\sigma_i)^2/2 + z_p \cdot \sigma_i) - d_i \\ &= d_i \cdot (\exp((-\sigma_i)^2/2 + z_p \cdot \sigma_i) - 1). \end{aligned} \quad (1)$$

3.3. Extended project network

As discussed above, the baseline schedule includes new precedence relationships caused by breaking potential resource contentions. The extended project network $\text{PN}(V, \tilde{E})$ consists of the new precedence relationships identified by the heuristic in Appendix C, the original precedence relationships and all tasks. When new precedence relationships are added, existing ones that are now implied by transitivity are removed. Based on the extended project network $\text{PN}(V, \tilde{E})$ with mean task durations, we carry out a standard critical path analysis and obtain, for each task i , the Earliest Start Time $\text{EST}(i)$, Latest Start Time $\text{LST}(i)$, Earliest Finish Time $\text{EFT}(i)$ and Latest Finish Time $\text{LFT}(i)$. The critical chain consists of tasks with zero total slack, i.e., those for which $\text{EFT}(i) = \text{LFT}(i)$. When there is more than one critical chain, we randomly choose one, and all other tasks form noncritical chains. Then, we calculate a late finish project schedule as an input to our buffer sizing procedure.

4. Buffer sizing procedure

This section describes our buffer sizing procedure. Fig. 4 illustrates the detailed procedure. Section 4.1 presents the network decomposition procedure. Based on the extended project network derived from critical chain identification, this procedure considers deterministic task durations and allows uncertainties to be absorbed by buffers. Section 4.2 describes feeding buffer sizing. Section 4.3 uses the decomposed network and the feeding buffers to determine the project buffer size.

4.1. Decomposition

Decomposition of the project network starts from the identified critical chain, and provides an understanding of the relationships between network structure and buffers. We denote critical and noncritical tasks by c and $\tilde{c}$, respectively. Suppose the critical chain for $\text{PN}(V, \tilde{E})$, excluding dummy start and end tasks, is $c_1 \rightarrow c_2 \rightarrow \dots \rightarrow c_q$, where q is the number of non-dummy critical tasks. For the start and finish time $\text{ST}(c_i)$ and $\text{FT}(c_i)$ of every critical task c_i , we have

$$\text{ST}(c_i) = \text{EST}(c_i) = \text{LST}(c_i), \quad \text{FT}(c_i) = \text{EFT}(c_i) = \text{LFT}(c_i),$$

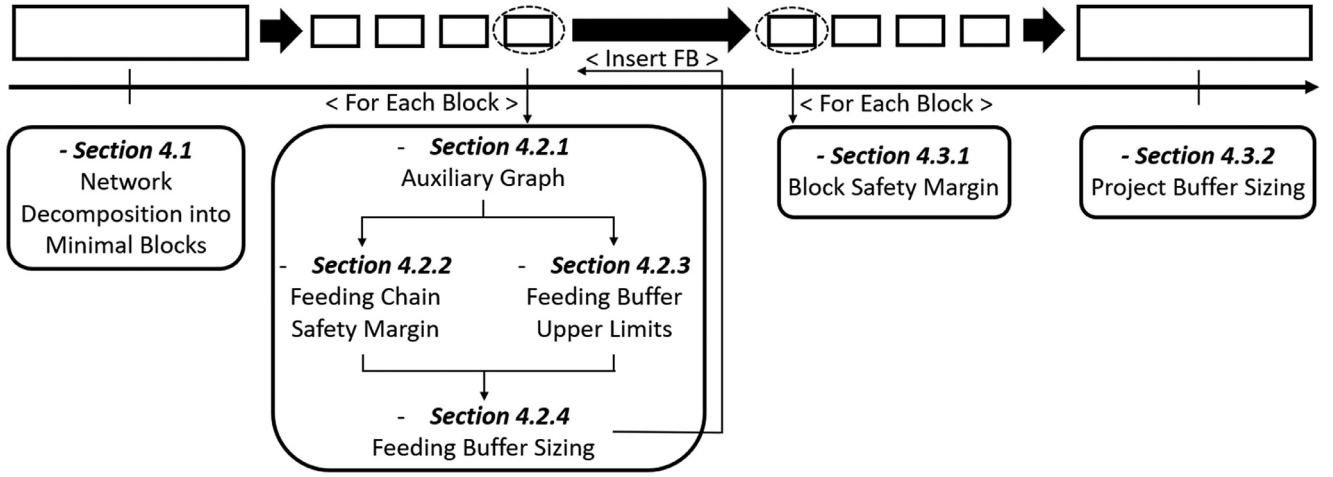


Fig. 4. Buffer sizing procedure illustration.

and

$$FT(c_i) = ST(c_{i+1}), \quad i = 1, 2, \dots, q, \quad (2)$$

where c_{q+1} is the dummy end task n .

Proposition 1. Given a critical chain, Eq. (2) indicates the redundancy of some precedence relationships between noncritical tasks. Specifically, for every three precedence relationships $(\tilde{c}_j, \tilde{c}_k)$, $(\tilde{c}_j, c_{i+1})$, and $(c_i, \tilde{c}_k)$, the precedence relationship $(\tilde{c}_j, \tilde{c}_k)$ is redundant.

Proof. The second and third precedence relationships imply

$$LFT(\tilde{c}_j) \leq ST(c_{i+1}), \quad FT(c_i) \leq EST(\tilde{c}_k)$$

Then, the critical chain relationship $FT(c_i) = ST(c_{i+1})$ implies

$$LFT(\tilde{c}_j) \leq EST(\tilde{c}_k).$$

Thus, the precedence relationship $(\tilde{c}_j, \tilde{c}_k)$ is redundant. $\square$

We name as *conditionally redundant* the precedence relationships that become redundant according to Proposition 1, since they depend on the critical chain that is identified. We delete these precedence relationships from $\tilde{E}$ for the purposes of network decomposition for a given critical chain, but retain them when forming a complete schedule. This deletion does not compromise task safety margins.

Based on the identified critical chain, we define the critical time interval and (minimal) block as follows for the decomposition procedure.

Definition 1. For every critical task c_i , the corresponding *critical time interval* is $CTI(c_i) = [ST(c_i), FT(c_i))$.

Definition 2. A *block* of the project network $PN(V, \tilde{E})$, represented by a *block time interval* $BTI = [ST(c_i), FT(c_j))$ with $c_i \leq c_j$, is a component of $PN(V, \tilde{E})$ that includes the partial critical chain $c_i \rightarrow \dots \rightarrow c_j$ and satisfies the following three conditions: (a) every task $v \in V$ in the component has $EST(v) \geq ST(c_i)$ and $LFT(v) \leq FT(c_j)$; (b) any task $v \in V$ outside the component has either $LFT(v) \leq ST(c_i)$ or $EST(v) \geq FT(c_j)$; and (c) every noncritical task in the component has no precedence relationship with any noncritical task outside the component. A *minimal block* is a block for which any of its sub-components in $PN(V, \tilde{E})$ is not a block.

Remark 1. Conditions (b) and (c) in Definition 2 guarantee that a block includes all relevant tasks and precedence relationships on which a feeding buffer depends. Therefore, the determination of feeding buffers can be made for each block individually.

Given the above definitions, we propose the following procedure to decompose the network $PN(V, \tilde{E})$ into minimal blocks. The algorithm starts by identifying critical time intervals and then builds up disjoint minimal block time intervals by relating tasks to them while guaranteeing that no immediate precedence relationship exists between any pair of noncritical tasks from different blocks.

Algorithm MB

- (0). *Initialization.* Compute $EST(v)$ and $LFT(v)$ for every non-dummy task $v \in V$, identify the critical chain and let $CTI(c_i) = [ST(c_i), FT(c_i))$, $i = 1, 2, \dots, q$.
- (1). *Compute initial time intervals.* For every non-dummy task $v \in V$, find the largest $ST(c_i) \leq EST(v)$ and the smallest $FT(c_j) \geq LFT(v)$. We say this task is *related* to the time interval $[ST(c_i), FT(c_j))$.
- (2). *Obtain disjoint time intervals.* Replace any two non-disjoint time intervals identified in Step (1) by their union. Correspondingly, the tasks related to the two intervals become related to their union. Repeat until all time intervals are disjoint.
- (3). *Obtain final time intervals.* If there is a precedence relationship connecting two noncritical tasks respectively related to two (disjoint) time intervals $[ST(c_i), FT(c_j))$ and $[ST(c_{i'}), FT(c_{j'}))$ with $FT(c_j) \leq ST(c_{i'})$, create a new time interval $[ST(c_i), FT(c_{j'}))$ to replace all time intervals that are its subsets. All relevant tasks are related to this new time interval. Repeat until no such precedence relationship connects two time intervals.
- (4). *Output BTIs and minimal blocks.* Each remaining BTI defines a minimal block consisting of all related tasks and the precedence relationships between them.

The Algorithm MB finds the minimal blocks of network $PN(V, \tilde{E})$ in $O(n^2)$ time as proved in Appendix E. The following remark shows the relations between the minimal blocks and the critical chain and the project network.

Remark 2. Each minimal block contains at least one critical task. Every pair of consecutive minimal blocks are connected by a precedence relationship between the last and first critical tasks, respectively, in the two blocks. Also, the sequence of all minimal blocks defines the extended project network without the conditionally redundant precedence relationships.

The example below illustrates the block definition and Algorithm MB using the adapted data of a real-life project C201211 [2,29,42], which is Provided in Table D.1 in Appendix D.

Table 1
Information for extended network of project C201211.

Tasks	2	5	8	11	12	3	4	6	7	9	10	13
ST	0	12	29	39	46	×	×	×	×	×	×	×
FT	12	29	39	46	52	×	×	×	×	×	×	×
EST	0	12	29	39	46	0	0	12	29	39	39	43
LFT	12	29	39	46	52	12	12	29	39	46	46	52

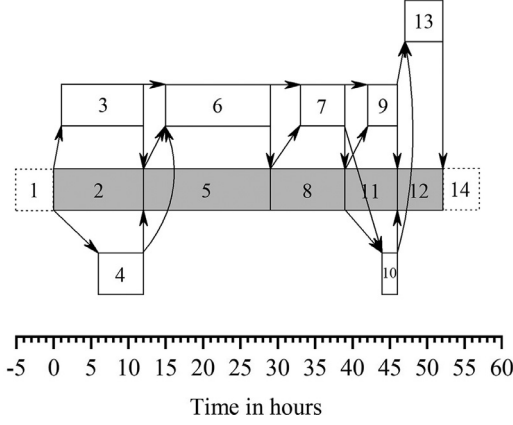


Fig. 5. Extended network of project C201211.

Example 1. Fig. 5 shows the network derived from the data of Project C201211. Tasks 1 and 14 are the dummy start and end, respectively. The critical chain is $2 \rightarrow 5 \rightarrow 8 \rightarrow 11 \rightarrow 12$. The precedence relationships are represented by arrows from the end of a predecessor task to the start of a successor task. Note there is also a precedence relationship between every two consecutive critical tasks.

Given the above critical chain, we first delete from the extended project network the conditionally redundant precedence relationships (3,6), (4,6), (6,7), (7,9), and (7,10). The network is then decomposed into four minimal blocks using Algorithm MB, as shown below.

(0). Table 1 shows the initial information. The CTIs are:

$$\begin{aligned} \text{CTI}(2) &= [\text{ST}(2), \text{FT}(2)] = [0, 12]; \\ \text{CTI}(5) &= [\text{ST}(5), \text{FT}(5)] = [12, 29]; \\ \text{CTI}(8) &= [\text{ST}(8), \text{FT}(8)] = [29, 39]; \\ \text{CTI}(11) &= [\text{ST}(11), \text{FT}(11)] = [39, 46]; \\ \text{CTI}(12) &= [\text{ST}(12), \text{FT}(12)] = [46, 52]. \end{aligned}$$

- For each task i , we compare $\text{EST}(i)$ and $\text{LFT}(i)$ with all CTIs to find the related time intervals. For example, for task 13 we have $\text{EST}(13) = 43$ and $\text{LFT}(13) = 52$. Then, $\text{ST}(11) = 39$ is the largest $\text{ST} \leq 43$ and $\text{FT}(12) = 52$ is the smallest $\text{FT} \geq 52$, hence task 13 is related to the time interval $[39, 52]$. The identified time intervals and their related tasks are: $[0, 12] \sim \{2, 3, 4\}$; $[12, 29] \sim \{5, 6\}$; $[29, 39] \sim \{8, 7\}$; $[39, 46] \sim \{11, 9, 10\}$; $[39, 52] \sim \{13\}$; $[46, 52] \sim \{12\}$.
- First, $[39, 46]$ and $[39, 52]$ are replaced by their union $[39, 52]$, and its related tasks are $\{11, 9, 10, 13\}$. Then, since intervals $[39, 52]$ and $[46, 52]$ are not disjoint, we replace them by $[39, 52]$, and its related tasks are $\{11, 12, 9, 10, 13\}$. The final disjoint time intervals are $[0, 12]$, $[12, 29]$, $[29, 39]$ and $[39, 52]$.
- For every two of these disjoint time intervals, there is no precedence relationship connecting noncritical tasks respectively related to them.
- The four final time intervals define four minimal blocks, as summarized in Table 2.

Table 2
Minimal blocks for project C201211.

	BTI	Tasks	Precedence relationships
Block 1	[0,12)	2,3,4	
Block 2	[12,29)	5,6	
Block 3	[29,39)	7,8	
Block 4	[39,52)	9, 10, 11, 12, 13	(9, 12), (9, 13), (10, 12), (10, 13), (11, 12)

4.2. Feeding buffers

Within all minimal blocks of network $\text{PN}(V, \tilde{E})$, where there is an immediate precedence relationship $(\tilde{c}_j, c_i)$ between noncritical and critical tasks, we add a feeding buffer $\text{FB}(\tilde{c}_j)$ to the end of task $\tilde{c}_j$ and replace $\tilde{c}_j$ with $\tilde{c}_j + \text{FB}(\tilde{c}_j)$ in every precedence relationship that includes $\tilde{c}_j$. The end time of $\text{FB}(\tilde{c}_j)$ is set to $\text{ST}(c_i)$, to construct a late-finish schedule. According to Remark 1, we first define an auxiliary graph $G(B)$ for every minimal block B , to describe the relative positions of feeding chains and feeding buffers and to prepare for buffer sizing, as described in Sections 4.2.2–4.2.4 and 4.3.

4.2.1. Auxiliary graph

The auxiliary graph $G(B) = (V(B), A(B))$ for block B is a directed graph with node set $V(B)$ and arc set $A(B)$ defined as follows. Node set $V(B)$ consists of two types of nodes:

- CTI nodes for the start time $\text{ST}(c_i)$ and end time $\text{FT}(c_i)$ of every critical task c_i in B . In particular, for any two consecutive critical tasks c_i and c_{i+1} , nodes $\text{FT}(c_i)$ and $\text{ST}(c_{i+1})$ are treated as one node and denoted by $\text{FT}(c_i) : \text{ST}(c_{i+1})$.
- Noncritical-task nodes for any noncritical task $\tilde{c}_j$ in B . In particular, if task $\tilde{c}_j$ has either no successor in B or a critical-task successor in B , then a feeding buffer $\text{FB}(\tilde{c}_j)$ is added to $\tilde{c}_j$ and the node is $\tilde{c}_j + \text{FB}(\tilde{c}_j)$.

Arc set $A(B)$ consists of three types of arcs:

- ArCs connecting CTI nodes: for every critical task c_i in B , there is an arc from node $\text{ST}(c_i)$ to node $\text{FT}(c_i)$.
- ArCs connecting CTI nodes and noncritical-task nodes. For each noncritical task $\tilde{c}_j$ in B :
 - If it has no predecessor in B , then there is an arc from the start-time node $\text{BTI}(B)$, which is a CTI node, to node $\tilde{c}_j$;
 - If it has a critical-task predecessor c_i in B , then there is an arc from node $\text{FT}(c_i)$ to node $\tilde{c}_j$;
 - If it has no successor in B , then it is node $\tilde{c}_j + \text{FB}(\tilde{c}_j)$. From this node there is an arc to the end-time node $\text{BTI}(B)$, which is a CTI node;
 - If it has a critical task successor c_i in B , then it is node $\tilde{c}_j + \text{FB}(\tilde{c}_j)$. From this node there is an arc to node $\text{ST}(c_i)$.
- ArCs connecting noncritical-task nodes, i.e., for every precedence relationship $(\tilde{c}_i, \tilde{c}_j)$ between noncritical tasks (with or without feeding buffers) in B , there is an arc from node $\tilde{c}_i$ (or $\tilde{c}_i + \text{FB}(\tilde{c}_i)$) to node $\tilde{c}_j$ (or $\tilde{c}_j + \text{FB}(\tilde{c}_j)$).

The example below constructs a block auxiliary graph of Project C201211.

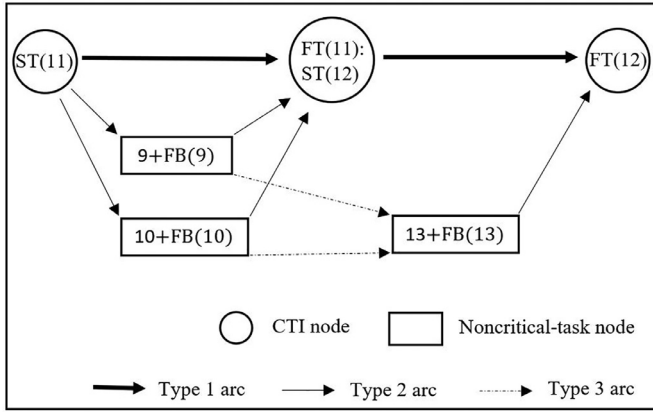


Fig. 6. $G(B_4)$ for block 4 of project C201211.

Example 2. We construct the auxiliary graph $G(B_4)$ for Block 4 of Project C201211, as shown in Fig. 6.

Based on the above definition of $G(B)$, we apply depth-first search by Cormen et al. [5, pp. 594–610] to find all feeding chains in $G(B)$. For each feeding buffer $FB(\tilde{c}_j)$, there is at least one feeding chain. Suppose there are W feeding chains, denoted by $FC_w(\tilde{c}_j)$, $w = 1, \dots, W$, feeding the buffer $FB(\tilde{c}_j)$. The size of every feeding buffer depends on two factors. One is the safety margins of its feeding chains, and the other is an upper limit that ensures the critical chain is not challenged by the insertion of feeding buffers. We denote these two factors, respectively, by feeding chain safety margins $FCSM$ and upper limits $\overline{FB}$.

4.2.2. Feeding chain safety margins

Feeding chain safety margins are calculated based on feeding chain properties. The feeding chains $FC_w(\tilde{c}_j)$ feeding the buffer $FB(\tilde{c}_j)$ have two properties: (a) each $FC_w(\tilde{c}_j)$ consists of only noncritical-task nodes, with or without a feeding buffer, and ends with node $\tilde{c}_j + FB(\tilde{c}_j)$; (b) for each $FC_w(\tilde{c}_j)$, the immediate predecessor of its start node and the immediate successor of its end node are CTI nodes in $G(B)$. We denote the two CTI nodes by $P_w(\tilde{c}_j)$ and $S(\tilde{c}_j)$. The successor CTI node for every chain $FC_w(\tilde{c}_j)$ is the same $S(\tilde{c}_j)$.

We denote the safety margin of chain $FC_w(\tilde{c}_j)$ by $FCSM_w(\tilde{c}_j)$. It aggregates safety margins of noncritical tasks on the chain using the square root of the sum of squares of these safety margins, which is a widely used approach in the literature for generating reliable buffer sizes [27,38,46,49]. Specifically, the following remark describes the calculation of feeding chain safety margins.

Remark 3. The calculation of $FC_w(\tilde{c}_j)$ involves two possible scenarios: (a) $FC_w(\tilde{c}_j)$ is a feeding chain with all nodes having no feeding buffer added, except for the end node $\tilde{c}_j + FB(\tilde{c}_j)$. Then, $FCSM_w(\tilde{c}_j)$ is defined as the square root of the sum of squared safety margins of all noncritical tasks, including $\tilde{c}_j$, in the chain. (b) $FC_w(\tilde{c}_j)$ is a feeding chain with more than one node having a feeding buffer added. Suppose in the chain the last feeding buffer preceding $FB(\tilde{c}_j)$ is $FB(\tilde{c}_k)$. The corresponding feeding chain $FC_u(\tilde{c}_k)$, where u denotes the serial number of feeding chains for $FB(\tilde{c}_k)$, is a part of $FC_w(\tilde{c}_j)$ starting from its first node and ending at $\tilde{c}_k + FB(\tilde{c}_k)$. Then, $FCSM_w(\tilde{c}_j)$ is defined as the square root of the sum of squared safety margins of tasks between $FB(\tilde{c}_k)$ and $FB(\tilde{c}_j)$, including $\tilde{c}_j$ and excluding $\tilde{c}_k$, in the chain.

Subsequently, we define the effective feeding chain given every feeding chain. This definition is helpful for understanding the calculation of feeding chain safety margins as well as determining the project buffer as described in Section 4.3.

Definition 3. Given every $FC_w(\tilde{c}_j)$, we define the chain of noncritical tasks determining $FCSM_w(\tilde{c}_j)$ as an *effective feeding chain* denoted by $EFC_w(\tilde{c}_j)$, and denote its immediate predecessor and successor CTI nodes in $G(B)$ by $P'_w(\tilde{c}_j)$ and $S'_w(\tilde{c}_j)$, respectively.

For scenario (a) in Remark 3, nodes $P'_w(\tilde{c}_j)$ and $S'_w(\tilde{c}_j)$ are $P_w(\tilde{c}_j)$ and $S(\tilde{c}_j)$, respectively. In scenario (b), since $EFC_w(\tilde{c}_j)$ has no immediate CTI predecessor in $G(B)$, we treat $S(\tilde{c}_k)$, i.e., the CTI successor of $FC_u(\tilde{c}_k)$, as $P'_w(\tilde{c}_j)$, while $S'_w(\tilde{c}_j)$ is $S(\tilde{c}_j)$.

We present an example of calculating feeding chain safety margins and defining effective feeding chains. Suppose there is a feeding chain $FC_w(\tilde{c}_j)$:

$$P_w(\tilde{c}_j) > \tilde{c}_k + FB(\tilde{c}_k) \rightarrow \tilde{c}_l \rightarrow \tilde{c}_j + FB(\tilde{c}_j) > S(\tilde{c}_j),$$

where its CTI predecessor and successor are separated by $>$ from the chain. The corresponding feeding chain of $FB(\tilde{c}_k)$ is

$$FC_u(\tilde{c}_k) : P_u(\tilde{c}_k) > \tilde{c}_k + FB(\tilde{c}_k) > S(\tilde{c}_k),$$

where $P_u(\tilde{c}_k) = P_w(\tilde{c}_j)$. Then, we have

$$FCSM_w(\tilde{c}_j) = \sqrt{(SM(\tilde{c}_l))^2 + (SM(\tilde{c}_j))^2}, \quad FCSM_u(\tilde{c}_k) = SM(\tilde{c}_k).$$

The effective feeding chains are respectively:

$$EFC_w(\tilde{c}_j) : S(\tilde{c}_k) > \tilde{c}_l \rightarrow \tilde{c}_j > S(\tilde{c}_j), \quad \text{and} \quad EFC_u(\tilde{c}_k) : P_u(\tilde{c}_k) > \tilde{c}_k > S(\tilde{c}_k).$$

4.2.3. Upper limits

To avoid the issue of a challenged critical chain, we define the upper limit $\overline{FB}(\tilde{c}_j)$ guaranteeing that every $FC_w(\tilde{c}_j)$ has a length not greater than $S(\tilde{c}_j) - P_w(\tilde{c}_j)$, i.e., the difference of time values of two CTI nodes, given mean estimates of the task durations. This generates the following constraints on $FB(\tilde{c}_j)$:

$$\sum_{\tilde{c}_k \text{ on } FC_w(\tilde{c}_j)} d_{\tilde{c}_k} + \sum_{FB(\tilde{c}_l) \text{ on } FC_w(\tilde{c}_j)} FB(\tilde{c}_l) \leq S(\tilde{c}_j) - P_w(\tilde{c}_j), \quad w = 1, \dots, W. \quad (3)$$

Since there can be other feeding buffers besides $FB(\tilde{c}_j)$ in some of its feeding chains, the upper limit $\overline{FB}(\tilde{c}_j)$ could depend on other feeding buffers. For example, suppose we have a chain $FC_w(\tilde{c}_j)$:

$$P_w(\tilde{c}_j) > \tilde{c}_k + FB(\tilde{c}_k) \rightarrow \tilde{c}_j + FB(\tilde{c}_j) > S(\tilde{c}_j),$$

where the corresponding feeding chain of $FB(\tilde{c}_k)$ is

$$FC_u(\tilde{c}_k) : P_u(\tilde{c}_k) > \tilde{c}_k + FB(\tilde{c}_k) > S(\tilde{c}_k).$$

Then, the two feeding buffers should satisfy the constraints

$$d_{\tilde{c}_k} + FB(\tilde{c}_k) \leq S(\tilde{c}_k) - P_u(\tilde{c}_k), \quad \text{and} \quad d_{\tilde{c}_k} + FB(\tilde{c}_k) + d_{\tilde{c}_j} + FB(\tilde{c}_j) \leq S(\tilde{c}_j) - P_w(\tilde{c}_j).$$

To calculate the upper limits for all feeding buffers within a block B , we solve a linear program $LP(B)$ with the objective of maximizing the sum of the feeding buffers, subject to constraints (3). The sizes of the feeding buffers are decision variables. Let $V(B, FB)$ denote the set of noncritical tasks to which feeding buffers are added. The model is

$$LP(B) \quad \max \quad \sum_{\tilde{c}_j \in V(B, FB)} FB(\tilde{c}_j) \\ \text{s.t.} \quad \text{Constraints of } FB(\tilde{c}_j), \quad \forall \tilde{c}_j \in V(B, FB), \\ FB(\tilde{c}_j) \geq 0, \quad \forall \tilde{c}_j \in V(B, FB). \quad (4)$$

An optimal solution to $LP(B)$ gives upper limits $\overline{FB}$ for feeding buffers in block B . The following theorem guarantees the existence of an optimal solution.

Theorem 1. For the linear program $LP(B)$, there exists a feasible finite optimal solution.

Proof. The standard form of LP(B) is

$$\begin{aligned} \text{LP}(B) \quad & \max \mathbf{1}^\top \mathbf{x} \\ \text{s.t.} \quad & \mathbf{A}\mathbf{x} \leq \mathbf{b}, \\ & \mathbf{x} \geq \mathbf{0}, \end{aligned}$$

where vector $\mathbf{x}$ denotes feeding buffers in B ; every element of matrix $\mathbf{A}$ is either 1 or 0 and each column of $\mathbf{A}$ has at least one element equal to 1, since each feeding buffer enters at least one of constraints (3); vector $\mathbf{b}$ derived from transformation of constraints (3) is nonnegative because the critical chain is the longest path. This problem is feasible since there is a feasible solution at $\mathbf{x} = \mathbf{0}$.

Its dual problem is

$$\begin{aligned} \text{Dual-LP}(B) \quad & \min \mathbf{b}^\top \mathbf{y} \\ \text{s.t.} \quad & \mathbf{A}^\top \mathbf{y} \geq \mathbf{1}, \\ & \mathbf{y} \geq \mathbf{0}, \end{aligned}$$

which has a feasible solution at $\mathbf{y} = \mathbf{1}$.

From strong duality, since LP(B) and its dual are both feasible, they both have feasible finite optimal solutions. $\square$

4.2.4. Feeding buffer size

The feeding chain safety margins and upper limits, as discussed above, determine the feeding buffer size as follows. Let

$$\text{FB}^0(\tilde{c}_j) = \max\{\text{FCSM}_w(\tilde{c}_j), w = 1, \dots, W\}.$$

Then, the size of $\text{FB}(\tilde{c}_j)$ is

$$\text{FB}^*(\tilde{c}_j) = \min\{\text{FB}^0(\tilde{c}_j), \overline{\text{FB}}(\tilde{c}_j)\}. \quad (5)$$

We understand it is common practice that task durations and start times are integers, in which case we need integer-valued feeding buffers, which are defined as

$$\text{FB}^\dagger(\tilde{c}_j) = \min\{\lceil \text{FB}^0(\tilde{c}_j) \rceil, \lfloor \overline{\text{FB}}(\tilde{c}_j) \rfloor\}.$$

The buffer size defined above guarantees that $\text{FB}^*(\tilde{c}_j)$ exceeds neither the upper limit $\overline{\text{FB}}(\tilde{c}_j)$, nor the maximal quantity of safety margin that should be covered. When $\text{FB}^*(\tilde{c}_j) = \overline{\text{FB}}(\tilde{c}_j) < \text{FB}^0(\tilde{c}_j)$, there exists a chain $\text{FC}_{\tilde{w}}(\tilde{c}_j)$, $1 \leq \tilde{w} \leq W$, for which $\text{FB}^*(\tilde{c}_j)$ covers only a part of its safety margin $\text{FCSM}_{\tilde{w}}(\tilde{c}_j)$. In this case, the remaining quantity $\text{FCSM}_{\tilde{w}}(\tilde{c}_j) - \text{FB}^*(\tilde{c}_j)$ should be included within the project buffer in addition to the safety margins of the critical tasks. We now present one of our main results.

Theorem 2. The insertion of feeding buffers defined in (5) keeps the identified critical chain unchallenged at the planning stage.

Proof. Eq. (5) establishes that the sizes of feeding buffers do not exceed the upper limits $\overline{\text{FB}}$. Constraints (3) further guarantee that feeding chains with insertion of such buffers do not exceed the identified critical chain in length, given mean task durations. Thus, the identified critical chain is not challenged at the planning stage. $\square$

The example below illustrates the feeding buffer sizing procedure for Block 4 of Project C201211.

Example 3. For Block 4, values of ST and FT are listed in Table 1; the feeding buffers, feeding chains, constraints of feeding buffers and feeding chain safety margins are listed in Table 3; and task durations and safety margins are provided in Table D.1.

To calculate the upper limits $\overline{\text{FB}}$, we solve the linear program $\text{LP}(B_4)$:

$$\begin{aligned} \text{LP}(B_4) \quad & \max \text{FB}(9) + \text{FB}(10) + \text{FB}(13) \\ \text{s.t.} \quad & \text{Constraints on FB}(9), \text{FB}(10), \text{and FB}(13), \\ & \text{FB}(9), \text{FB}(10), \text{FB}(13) \geq 0. \end{aligned}$$

Table 3

FBs, FCs, EFCs, constraints of FBs and FCSMs for block 4.

FB	FB(9)
FC	$\text{FC}_1(9) : \text{ST}(11) > 9 + \text{FB}(9) > \text{FT}(11)$
EFC	$\text{EFC}_1(9) : \text{ST}(11) > 9 > \text{FT}(11)$
Constraints of FB	$d_9 + \text{FB}(9) \leq \text{FT}(11) - \text{ST}(11)$
FCSM	$\text{FCSM}_1(9) = \text{SM}(9) = 0.92$
FB	FB(10)
FC	$\text{FC}_1(10) : \text{ST}(11) > 10 + \text{FB}(10) > \text{FT}(11)$
EFC	$\text{EFC}_1(10) : \text{ST}(11) > 10 > \text{FT}(11)$
Constraints of FB	$d_{10} + \text{FB}(10) \leq \text{FT}(11) - \text{ST}(11)$
FCSM	$\text{FCSM}_1(10) = \text{SM}(10) = 0.46$
FB	FB(13)
FC	$\text{FC}_1(13) : \text{ST}(11) > 9 + \text{FB}(9) \rightarrow 13 + \text{FB}(13) > \text{FT}(12)$
	$\text{FC}_2(13) : \text{ST}(11) > 10 + \text{FB}(10) \rightarrow 13 + \text{FB}(13) > \text{FT}(12)$
EFC	$\text{EFC}_1(13) : \text{FT}(11) > 13 > \text{FT}(12)$
	$\text{EFC}_2(13) : \text{FT}(11) > 13 > \text{FT}(12)$
Constraints of FB	$d_9 + \text{FB}(9) + d_{13} + \text{FB}(13) \leq \text{FT}(12) - \text{ST}(11)$
	$d_{10} + \text{FB}(10) + d_{13} + \text{FB}(13) \leq \text{FT}(12) - \text{ST}(11)$
FCSM	$\text{FCSM}_1(13) = \text{SM}(13) = 1.15$
	$\text{FCSM}_2(13) = \text{SM}(13) = 1.15$

The optimal solution is $\overline{\text{FB}}(9) = 3$, $\overline{\text{FB}}(10) = 5$, $\overline{\text{FB}}(13) = 1$. Then, we have

$$\text{FB}^*(9) = \min\{\text{FB}^0(9), \overline{\text{FB}}(9)\} = \min\{\text{SM}(9), 3\} = 0.92 \quad (1),$$

$$\text{FB}^*(10) = \min\{\text{FB}^0(10), \overline{\text{FB}}(10)\} = \min\{\text{SM}(10), 5\} = 0.46 \quad (1),$$

$$\text{FB}^*(13) = \min\{\text{FB}^0(13), \overline{\text{FB}}(13)\} = \min\{\text{SM}(13), 1\} = 1 \quad (1),$$

where the integer-valued feeding buffers are shown in the brackets.

4.3. Project buffer

Given that all feeding buffers have been sized, we now determine the project buffer to protect the critical chain and the overall project. According to Remark 2, the project buffer integrates block safety margins, which are calculated first in Section 4.3.1. Then, the project buffer size is given in Section 4.3.2.

4.3.1. Block safety margin

A *block safety margin* is the difference between the safe estimate of completion time of all tasks in a block and the expected length of the critical chain included in the block. Therefore, the safety margin $\text{BSM}(B)$ for minimal block B covers both the safety margins of critical tasks and the remaining safety margins of effective feeding chains defined as follows.

Definition 4. The *remaining safety margin* (RSM) of an effective feeding chain $\text{EFC}_w(\tilde{c}_j)$ is the safety margin of this chain that is not covered by $\text{FB}^*(\tilde{c}_j)$, i.e., $\text{RSM}_w(\tilde{c}_j) = (\text{FCSM}_w(\tilde{c}_j) - \text{FB}^*(\tilde{c}_j))^+$, where $\text{FCSM}_w(\tilde{c}_j)$ is calculated according to Remark 3.

Wherever there is a feeding chain $\text{EFC}_w(\tilde{c}_j)$, an $\text{RSM}_w(\tilde{c}_j)$ is defined. For example, consider the feeding chains

$$\text{FC}_w(\tilde{c}_j) : P_w(\tilde{c}_j) > \tilde{c}_k + \text{FB}(\tilde{c}_k) \rightarrow \tilde{c}_l \rightarrow \tilde{c}_j + \text{FB}(\tilde{c}_j) > S(\tilde{c}_j),$$

$$\text{FC}_u(\tilde{c}_k) : P_u(\tilde{c}_k) > \tilde{c}_k + \text{FB}(\tilde{c}_k) > S(\tilde{c}_k).$$

The corresponding effective feeding chains are

$$\text{EFC}_w(\tilde{c}_j) : S(\tilde{c}_k) > \tilde{c}_l \rightarrow \tilde{c}_j > S(\tilde{c}_j),$$

$$\text{EFC}_u(\tilde{c}_k) : P_u(\tilde{c}_k) > \tilde{c}_k > S(\tilde{c}_k).$$

Then we have

$$\text{RSM}_w(\tilde{c}_j) = (\text{FCSM}_w(\tilde{c}_j) - \text{FB}^*(\tilde{c}_j))^+,$$

$$\text{RSM}_u(\tilde{c}_k) = (\text{FCSM}_u(\tilde{c}_k) - \text{FB}^*(\tilde{c}_k))^+.$$

Then, we define the critical counterpart of each effective feeding chain to enable comparisons of safety margins between critical tasks and noncritical chains for developing an algorithm to calculate the block safety margin $BSM(B)$ accurately.

Definition 5. We define the chain consisting of critical tasks that are processed concurrently with $EFC_w(\tilde{c}_j)$ as the *critical counterpart* of $EFC_w(\tilde{c}_j)$, and denote it by $CCP_w(\tilde{c}_j)$. Specifically, this is the chain of critical tasks forming the time interval $[P'_w(\tilde{c}_j), S'_w(\tilde{c}_j)]$, as described in Definition 3.

The algorithm below determines block safety margins covering both safety margins of critical tasks and remaining safety margins from noncritical chains. Adding all the $RSM_w(\tilde{c}_j)$ values to the block safety margin would create an unreasonably large project buffer. Therefore, we compare the safety margin of the critical counterpart $CCP_w(\tilde{c}_j)$ with $RSM_w(\tilde{c}_j)$ considering concurrent processing, and incorporate only the greater value. If a critical counterpart corresponds to different effective feeding chains, we compare it with all related remaining safety margins and incorporate only the maximal value.

Algorithm BSM

- (0). For each $FB(\tilde{c}_j)$ in B , we calculate the initial values:
 $EFC_w(\tilde{c}_j)$, $RSM_w(\tilde{c}_j)$, $P'_w(\tilde{c}_j)$, $S'_w(\tilde{c}_j)$, $CCP_w(\tilde{c}_j)$, $w = 1, \dots, W$.
- (1). Find all EFCs whose critical counterpart consists of a single critical task c_i . For each c_i , we define

$$\overline{SM}(c_i) = \max \{ SM(c_i), \\ RSMs \text{ of all EFCs corresponding to } c_i \}.$$

For every other critical task c_i in B , we define $\overline{SM}(c_i) = SM(c_i)$.

- (2). For every other $EFC_w(\tilde{c}_j)$ for which the critical counterpart consists of at least two critical tasks, we define $BSM_w(B, \tilde{c}_j) = \max\{BSM^0(B), BSM^1(B)\}$, where

$$BSM^0(B) = \sqrt{\sum_{c_i \in B} (\overline{SM}(c_i))^2},$$

$$BSM^1(B) = \sqrt{\sum_{c_i \in \tilde{B}} (\overline{SM}(c_i))^2 + (RSM_w(\tilde{c}_j))^2},$$

and $c_i \in B$ denotes critical tasks in B , while $c_i \in \tilde{B}$ denotes those critical tasks in B with CTIs outside the time interval $[P'_w(\tilde{c}_j), S'_w(\tilde{c}_j)]$.

- (3). Finally, we have

$$BSM(B) = \max\{BSM_w(B, \tilde{c}_j), \\ \text{for all } EFC_w(\tilde{c}_j) \text{ as described in Step (2)}\}.$$

In particular, when there is no such $EFC_w(\tilde{c}_j)$, we have

$$BSM(B) = BSM^0(B) = \sqrt{\sum_{c_i \in B} (\overline{SM}(c_i))^2}.$$

Let $\#(B)$ denote the number of effective feeding chains in block B . Then, the time complexity of Algorithm BSM is $O(\#(B))$ as proved in Appendix F.

4.3.2. Project buffer size

Finally, we determine the project buffer, given all block safety margins in the network $PN(V, \tilde{E})$ obtained in Section 4.3.1. Let $\tilde{m}$ be the number of minimal blocks, then the project buffer size is

$$PB^* = \sqrt{\sum_{i=1}^{\tilde{m}} (BSM(B_i))^2}. \quad (6)$$

Accordingly, the integer-valued project buffer is $PB^\dagger = \lceil PB^* \rceil$.

From Algorithm BSM and (6), we see that PB^* is at least $\sqrt{\sum_{i=1}^q (SM(c_i))^2}$, which is the total of safety margins of the critical

Table 4
Information for determining $BSM(B_4)$.

EFC		RSM	P'	S'	CCP
EFC ₁ (9)	9	$(FCSM_1(9) - FB^*(9))^+ = 0$	ST(11)	FT(11)	11
EFC ₁ (10)	10	$(FCSM_1(10) - FB^*(10))^+ = 0$	ST(11)	FT(11)	11
EFC ₁ (13)	13	$(FCSM_1(13) - FB^*(13))^+ = 0.15$	ST(12)	FT(12)	12
EFC ₂ (13)	13	$(FCSM_2(13) - FB^*(13))^+ = 0.15$	ST(12)	FT(12)	12

tasks. By integrating block safety margins, the project buffer covers both the safety margins of the critical chain and the remaining safety margins of the noncritical chains.

We now summarize the time complexity of the whole buffer sizing procedure. Let $\mathcal{L}(n_1, n_2)$ denote the time required to solve a linear program with n_1 constraints and n_2 variables. Then, for a project $PN(V, E)$, the time complexity of buffer sizing is $O(T_{\max} n^2 \log n + \#(PN) + \mathcal{L}(\#(PN), n))$, where $T_{\max} = \sum_{i=1}^n d_i$ and $\#(PN)$ is the number of effective feeding chains given a critical chain. The proof is in Appendix G.

For a single project with 150 tasks, the running time of our procedure with MATLAB R2018a is less than 10 seconds on a personal computer with processor Intel(R) Core(TM) i5-7400 CPU @3.00 GHZ and installed RAM of 8.00 GB. Since the procedure is used at the planning stage of projects, this computation time is small enough for practical use.

The example below describes the calculation of block safety margins via Algorithm BSM and the calculation of the project buffer for Project C201211.

Example 4. We illustrate the block safety margin decision procedure using Block 4 in Fig. 6, and then determine the project buffer for Project C201211. For Step (0), the initial values are provided in Table 4.

In Step (1), for the two critical tasks 11 and 12 in the block, we define

$$\overline{SM}(11) = \max\{SM(11), RSM_1(9), RSM_1(10)\} = 1.61,$$

$$\overline{SM}(12) = \max\{SM(12), RSM_1(13), RSM_2(13)\} = 1.38.$$

In Step (2), there is no effective feeding chain that corresponds to a critical counterpart with at least two tasks. In Step (3), we have

$$BSM(B_4) = \sqrt{(\overline{SM}(11))^2 + (\overline{SM}(12))^2} = 2.12.$$

In Step (4), we calculate the other block safety margins for the project:

$$BSM(B_1) = 2.77, \quad BSM(B_2) = 3.92, \quad BSM(B_3) = 2.31.$$

The project buffer is

$$PB^* = \sqrt{\sum_{i=1}^4 (BSM(B_i))^2} = 5.73,$$

and the integer-valued project buffer size is 6.

5. Experimental design

We design experiments to test the performance of our procedure against several benchmark methods from the literature. Section 5.1 provides definitions and motivations for our four performance indicators. Section 5.2 provides details of the five benchmark methods we use. Section 5.3 explains the generation of random data and our simulation procedure.

5.1. Performance indicators

Once buffer sizing is completed, we output a late-finish CCPM schedule by applying a critical path analysis to the extended project network with feeding and project buffers inserted. Let the

estimated project makespan be the duration of the baseline schedule amended by the addition of feeding buffers plus the project buffer, whereas the real project makespan is obtained via simulation. We define the following four performance indicators for the CCPM schedule.

- P_1 (MAD): the accuracy of the estimated project makespan, which is the mean absolute percentage error of the estimated project makespan from the real project makespan.
- P_2 (REL): the reliability of the estimated project makespan, which is the probability that the real project makespan does not exceed the estimated project makespan.
- P_3 (AFB): the average feeding buffer size, which is the sum of feeding buffer sizes divided by the number of feeding buffers.
- P_4 (IND): a binary indicator, with $P_4 = 1$ if the estimated project makespan is greater than the length of the critical chain given mean task durations plus the project buffer, and $P_4 = 0$ if they are equal.

P_1 measures how far the estimated project makespan is from the real project makespan. The closer P_1 is to zero, the more accurate is the estimate on average. Specifically, the percentage absolute error of the estimated project makespan (EPM) from the real project makespan (RPM) is calculated by

$$\frac{100|EPM - RPM|}{RPM} \%$$

P_2 measures the reliability of the estimated project makespan. However, the project company will use the estimated project makespan to bid competitively for contracts and to respond to requests for project work. Hence, a P_2 value of 1 is not ideal, because it indicates that a lower estimate would still have delivered the project on time with high reliability, while a higher estimate is likely to lead to a loss of competitive contracts. On the other hand, a low value of P_2 indicates that completing the project on time will be difficult, and may require expensive emergency procedures. Moreover, there is a considerable risk of late delivery, which may incur contract penalties or even hurt the reputation of the project company. Most companies would expect a probability of on-time completion, or *service level*, between 0.6 and 0.9 [36], but this varies between projects. For example, a project with high strategic value might be undertaken with a lower service level. It is the role of the project buffer to deliver a service level in the correct range.

Regarding the use of P_3 , we observe that, given an extended project network with the identified critical chain, the number of feeding buffers is fixed. This is because a feeding buffer is added wherever a noncritical task immediately precedes a critical task. Therefore, a smaller value of P_3 indicates two benefits. First, smaller feeding buffers lead to an unchallenged critical chain and help reduce the cost of rescheduling when the critical chain is challenged. Second, since insertion of feeding buffers moves non-critical tasks earlier in the project schedule, smaller buffers move tasks less and lead to later start times. Therefore, smaller feeding buffers have additional benefits in delivering late-start schedules and controlling work-in-progress [22, p. 133], delaying costs, and reducing rework since better information is available about the project [41, p. 195].

According to Theorem 2 and the definition of P_4 , a value of zero indicates the critical chain is unchallenged, which is the case for the proposed buffer sizing procedure, and a value of one indicates the opposite.

While we can directly calculate P_3 and P_4 once the project and feeding buffers are determined, we need to calculate P_1 and P_2 via simulation. Given d_i and σ_i for all tasks, we generate random task durations from lognormal distributions, as described in Section 3.2. The real project makespan is calculated using critical path analysis

over the extended project network with randomly generated log-normal task durations.

5.2. Benchmark methods

We compare our buffer sizing procedure, which we call PP, with the following five benchmarks using performance indicators P_1 , P_2 , P_3 and P_4 :

- Cut and Paste Method (C&PM, Goldratt [11]),
- Root Square Error Method (RSEM, Newbold [27]),
- Adaptive Procedure with Density (APD, Tukel et al. [38]),
- Monte Carlo simulation method (SMC, Tenera [35]),
- Method of Yu et al. [45].

The C&PM and RSEM are the most representative buffer sizing methods used as benchmarks in the literature. The APD represents methods that extend RSEM by scaling up buffer size with a global network complexity measure. Some other methods extend RSEM by incorporating measures of resource tightness, environmental, uncertainty or rework risks, but are rarely used as benchmarks. The SMC represents buffer sizing methods that use simulation without considering network structure. The Yu2013 method represents papers that determine buffer size using free floats, to avoid the issue of a challenged critical chain.

In the C&PM method [11], a project or feeding buffer is defined as half of the sum of safety margins of all tasks on the critical or longest feeding chain. Since we assume the given task durations are mean estimates, in C&PM we directly apply task safety margins calculated in Section 3.2. Further, as suggested by Newbold [27, p. 95], when there is only one task in a feeding chain, we define the safety margin of this task as the feeding buffer. The RSEM method defines the buffer size as the square root of the sum of squared safety margins of tasks on the associated chain. The APD method determines feeding buffer size based on variances of log-normal task durations, VAR, and a network complexity measure K . For every feeding buffer, K is defined as one plus the total number of precedence relationships on the sub-network feeding into the critical chain divided by the total number of tasks on the sub-network. Then, the buffer size equals K times the square root of the sum of VARs of tasks on the longest feeding chain. Tukel et al. [38] use a project buffer consisting of 0% or 20% of the estimated project makespan for experiments. According to their results, the latter tends to result in unnecessarily high reliability measures, i.e., $P_2 > 0.9$, indicating large estimated project makespans that would lead to a loss of competitive contracts, as discussed in Section 5.1; thus, we assume the project buffer size provided by APD is zero.

In the SMC method, the project buffer size is the difference between the simulated end time and the scheduled end time of the last task in the critical chain. Then, the feeding buffer size is the difference between the simulated end time and the scheduled end time of the task to which it is added. The simulated end time is associated with the p -valued completion ratio, corresponding to the p -level safety margin described in Section 3.2. The scheduled end time is calculated based on late finish times, given mean estimated task durations in the project network without buffers inserted. The method of Yu et al. [45] uses free slack and safety margin to determine buffer size. Each feeding buffer is the smaller of the free slack of the task to which it is added and the integrated safety margin of the longest feeding chain. In this case, the project buffer is found by integrating the safety margins of tasks on the critical chain plus the sum of remaining safety margins of all feeding chains, i.e., the positive differences between the integrated safety margin and the feeding buffer size. Since they do not decompose the network, all five benchmark methods operate on the whole project network.

Table 5
Comparative performance for project C201211.

C201211		P_1 (MAD): 100%				P_2 (REL): [0,1]				P_3 (AFB)				P_4 (IND): {0,1}			
		$p=0.6$	$p=0.7$	$p=0.8$	$p=0.9$	$p=0.6$	$p=0.7$	$p=0.8$	$p=0.9$	$p=0.6$	$p=0.7$	$p=0.8$	$p=0.9$	$p=0.6$	$p=0.7$	$p=0.8$	$p=0.9$
$\sigma=0.1$	PP	0.04	0.04	0.05	0.06	0.53	0.65	0.76	0.89	0.14	0.33	0.56	0.84	0	0	0	0
	C&PM	0.04	0.04	0.05	0.09	0.53	0.66	0.77	0.97	0.29	0.69	1.17	1.85	0	0	0	1
	RSEM	0.04	0.04	0.05	0.09	0.53	0.65	0.76	0.97	0.33	0.77	1.31	2.09	0	0	0	1
	APD	0.07	0.07	0.07	0.07	0.91	0.91	0.91	0.91	2.69	2.69	2.69	2.69	1	1	1	1
	SMC	0.04	0.04	0.05	0.12	0.60	0.68	0.83	1.00	0.75	1.22	1.81	2.71	0	0	1	1
$\sigma=0.2$	Yu2013	0.04	0.05	0.07	0.12	0.59	0.78	0.91	1.00	0.27	0.64	1.09	1.60	0	0	0	0
	PP	0.07	0.07	0.08	0.11	0.43	0.55	0.70	0.86	0.21	0.61	0.99	1.40	0	0	0	0
	C&PM	0.07	0.07	0.12	0.27	0.44	0.56	0.87	1.00	0.44	1.25	2.26	3.77	0	0	1	1
	RSEM	0.07	0.07	0.12	0.31	0.43	0.55	0.88	1.00	0.50	1.41	2.55	4.25	0	0	1	1
	APD	0.31	0.31	0.31	0.31	1.00	1.00	1.00	1.00	5.42	5.42	5.42	5.42	1	1	1	1
$\sigma=0.3$	SMC	0.08	0.14	0.29	0.50	0.70	0.93	1.00	1.00	2.12	3.12	4.43	6.17	1	1	1	1
	Yu2013	0.07	0.08	0.15	0.37	0.49	0.69	0.94	1.00	0.41	1.17	1.85	2.23	0	0	0	0
	PP	0.10	0.10	0.12	0.16	0.37	0.48	0.63	0.83	0.22	0.77	1.31	1.75	0	0	0	0
	C&PM	0.10	0.11	0.19	0.50	0.37	0.53	0.89	1.00	0.45	1.68	3.26	5.72	0	1	1	1
	RSEM	0.10	0.11	0.22	0.56	0.37	0.53	0.92	1.00	0.50	1.89	3.67	6.44	0	1	1	1
$\sigma=0.4$	APD	0.57	0.57	0.57	0.57	1.00	1.00	1.00	1.00	8.23	8.23	8.23	8.23	1	1	1	1
	SMC	0.18	0.35	0.60	1.00	0.86	0.99	1.00	1.00	3.45	5.10	7.18	10.56	1	1	1	1
	Yu2013	0.10	0.12	0.26	0.67	0.40	0.63	0.96	1.00	0.42	1.49	2.20	2.35	0	0	0	0
	PP	0.14	0.13	0.15	0.21	0.31	0.43	0.57	0.79	0.15	0.88	1.47	2.11	0	0	0	0
	C&PM	0.14	0.14	0.27	0.71	0.31	0.50	0.90	1.00	0.31	1.96	4.14	7.66	0	1	1	1
$\sigma=0.5$	RSEM	0.14	0.14	0.31	0.80	0.31	0.50	0.94	1.00	0.34	2.21	4.66	8.63	0	1	1	1
	APD	0.82	0.83	0.82	0.82	1.00	1.00	1.00	1.00	11.18	11.18	11.18	11.18	1	1	1	1
	SMC	0.32	0.57	0.91	1.42	0.94	1.00	1.00	1.00	4.94	7.17	10.17	14.57	1	1	1	1
	Yu2013	0.14	0.15	0.37	0.95	0.33	0.58	0.97	1.00	0.28	1.66	2.25	2.46	0	0	0	0
	PP	0.17	0.17	0.18	0.26	0.28	0.38	0.53	0.77	0.01	0.92	1.60	2.44	0	0	0	0
$\sigma=0.5$	C&PM	0.17	0.17	0.35	0.92	0.28	0.45	0.88	1.00	0.02	2.08	4.87	9.55	0	1	1	1
	RSEM	0.17	0.17	0.39	1.03	0.28	0.45	0.92	1.00	0.03	2.34	5.49	10.76	0	1	1	1
	APD	1.08	1.08	1.08	1.09	1.00	1.00	1.00	1.00	14.30	14.30	14.30	14.30	1	1	1	1
	SMC	0.44	0.74	1.15	1.76	0.95	1.00	1.00	1.00	6.07	8.87	12.50	18.13	1	1	1	1
	Yu2013	0.17	0.18	0.46	1.22	0.28	0.50	0.96	1.00	0.02	1.74	2.30	2.58	0	0	0	0

5.3. Data and simulation

In addition to the real-life Project C201211, we use RanGen2 software [7,28,40,42,43] to generate random project networks. In the generator, there are five input parameters: number of tasks, I_2 (the serial/parallel network indicator), number of resources, resource usage metric RU and resource scarceness metric RC. The I_2 value measures the closeness of a network to a serial or parallel network, with $I_2 = 0$ indicating that all tasks are in parallel and $I_2 = 1$ indicating that the network is completely serial. The resource usage metric RU is the number of resource types each task requests. The resource scarceness metric RC is the total demand of a resource divided by the product of the number of tasks requesting the resource and the availability of the resource.

We generate random project networks with 150 non-dummy tasks each, and set $I_2 \in \{0.25, 0.5, 0.75\}$. Since all buffer sizing procedures should start from the same extended project network with resource contentions previously solved by adding new precedence relationships, we directly set the number of resources as 1, $RU = 0$, and $RC = 0$ for the resource parameters to generate such extended networks without resource contentions. For each set of these parameters, we generate 30 random project networks. Therefore, there are 90 project networks in total. We treat the task durations generated as mean values. We assume that the lognormal standard deviations of all task duration variables in a project have the same value σ . According to Ashtiani et al. [1], the lognormal standard deviation σ can be reasonably assigned to a value in $(0, 0.5]$. In our buffer sizing simulations, we vary the value of $\sigma \in \{0.1, 0.2, 0.3, 0.4, 0.5\}$ and $p \in \{0.6, 0.7, 0.8, 0.9\}$. The value of p refers to the $p \cdot 100\%$ -confidence level when calculating the task safety margins within PP, C&PM, RSEM and Yu2013, and the buffer size within SMC. When applying the SMC method to determine the buffer size, we randomly generate 1000 replications of task durations for each project network. To calculate P_1 and P_2 via simulation for all methods, we also ran-

Table 6

Decomposition results for random project networks.

Set	I_2	Average number of MBs	Average number of tasks per MB
1	0.25	15.5	9.71
2	0.50	24.8	6.08
3	0.75	72.97	2.07

domly generate 1000 replications of task durations for each project network.

6. Computational study

Section 6.1 discusses the results of the procedure PP and five benchmarks applied to Project C201211. Section 6.2 discusses the results of all methods applied to 90 projects with random data generated by RanGen2.

6.1. Case study

For the Project C201211, Table 5 summarizes the results regarding the four performance indicators described in Section 5.1, and different values of σ and confidence levels of p .

The results in Table 5 provide the following observations:

- Our buffer sizing procedure PP generally provides smaller values of P_1 , indicating more accurate project makespan estimates, than the benchmarks. As p and σ increase, these differences become greater. For $p = 0.9$ and $\sigma = 0.5$, our procedure delivers a value of 0.26, whereas all other methods give values around 1 and the SMC presents the worst number of 1.76.
- Regarding P_2 , a value between $[0.6, 0.9]$ is preferred, as discussed in Section 5.1. For $p = 0.9$, procedure PP performs the best with reasonable values of P_2 ranging from 0.77 to 0.89, while all the other five methods produce values of P_2 that are

Table 7
Comparative performance on projects with $I_2 = 0.25$.

$I_2=0.25$		P'_1 (MAD): 100%				P'_2 (REL): [0,1]				P'_3 (AFB)				P'_4 (No. of Projects with IND=1)			
		$p=0.6$	$p=0.7$	$p=0.8$	$p=0.9$	$p=0.6$	$p=0.7$	$p=0.8$	$p=0.9$	$p=0.6$	$p=0.7$	$p=0.8$	$p=0.9$	$p=0.6$	$p=0.7$	$p=0.8$	$p=0.9$
$\sigma=0.1$	PP	0.01	0.01	0.01	0.02	0.34	0.45	0.58	0.76	0.14	0.32	0.54	0.84	0	0	0	0
	C&PM	0.01	0.03	0.06	0.11	0.58	0.93	1.00	1.00	0.32	0.75	1.27	2.02	30	30	30	30
	RSEM	0.01	0.01	0.02	0.05	0.41	0.65	0.90	1.00	0.28	0.65	1.11	1.76	30	30	30	30
	APD	0.37	0.37	0.37	0.37	1.00	1.00	1.00	1.00	5.24	5.24	5.24	5.24	30	30	30	30
	SMC	0.07	0.11	0.17	0.26	1.00	1.00	1.00	1.00	2.51	3.37	4.40	5.84	30	30	30	30
$\sigma=0.2$	Yu2013	0.01	0.03	0.06	0.13	0.62	0.93	1.00	1.00	0.24	0.57	0.95	1.45	0	0	0	0
	PP	0.03	0.03	0.03	0.03	0.21	0.31	0.45	0.67	0.21	0.58	1.01	1.56	0	0	0	0
	C&PM	0.03	0.05	0.13	0.28	0.38	0.88	1.00	1.00	0.48	1.37	2.47	4.12	30	30	30	30
	RSEM	0.03	0.03	0.05	0.14	0.26	0.55	0.92	1.00	0.42	1.19	2.15	3.58	30	30	30	30
	APD	0.86	0.86	0.86	0.86	1.00	1.00	1.00	1.00	10.55	10.55	10.55	10.55	30	30	30	30
$\sigma=0.3$	SMC	0.29	0.41	0.56	0.79	1.00	1.00	1.00	1.00	6.55	8.28	10.34	13.25	30	30	30	30
	Yu2013	0.03	0.05	0.15	0.37	0.40	0.90	1.00	1.00	0.37	1.01	1.72	2.58	0	0	0	0
	PP	0.06	0.05	0.04	0.04	0.14	0.21	0.34	0.57	0.21	0.77	1.38	2.16	0	0	0	0
	C&PM	0.05	0.06	0.19	0.47	0.22	0.80	1.00	1.00	0.49	1.84	3.56	6.24	30	30	30	30
	RSEM	0.05	0.04	0.08	0.24	0.16	0.44	0.92	1.00	0.42	1.60	3.10	5.44	30	30	30	30
$\sigma=0.4$	APD	1.35	1.35	1.35	1.35	1.00	1.00	1.00	1.00	16.03	16.03	16.03	16.03	30	30	30	30
	SMC	0.63	0.83	1.08	1.44	1.00	1.00	1.00	1.00	11.71	14.39	17.63	22.24	30	30	30	30
	Yu2013	0.05	0.06	0.25	0.72	0.23	0.85	1.00	1.00	0.37	1.33	2.32	3.40	0	0	0	0
	PP	0.09	0.07	0.06	0.05	0.09	0.14	0.25	0.47	0.14	0.88	1.68	2.66	0	0	0	0
	C&PM	0.08	0.06	0.25	0.64	0.12	0.67	1.00	1.00	0.33	2.14	4.52	8.36	30	30	30	30
$\sigma=0.5$	RSEM	0.09	0.06	0.10	0.34	0.10	0.32	0.89	1.00	0.29	1.86	3.93	7.28	30	30	30	30
	APD	1.82	1.82	1.82	1.82	1.00	1.00	1.00	1.00	21.76	21.76	21.76	21.76	30	30	30	30
	SMC	1.02	1.30	1.64	2.16	1.00	1.00	1.00	1.00	17.77	21.50	26.00	32.60	30	30	30	30
	Yu2013	0.08	0.07	0.35	1.12	0.12	0.74	1.00	1.00	0.25	1.52	2.76	3.93	0	0	0	0
	PP	0.12	0.11	0.08	0.07	0.06	0.10	0.19	0.40	0.01	0.93	1.91	3.06	0	0	0	0
$\sigma=0.5$	C&PM	0.12	0.07	0.28	0.79	0.06	0.51	0.99	1.00	0.03	2.27	5.32	10.42	30	30	30	30
	RSEM	0.12	0.08	0.11	0.43	0.06	0.23	0.85	1.00	0.02	1.98	4.63	9.08	30	30	30	30
	APD	2.29	2.29	2.29	2.29	1.00	1.00	1.00	1.00	27.84	27.84	27.84	27.84	30	30	30	30
	SMC	1.43	1.79	2.24	2.91	1.00	1.00	1.00	1.00	24.53	29.38	35.39	44.34	30	30	30	30
	Yu2013	0.12	0.07	0.43	1.54	0.06	0.57	1.00	1.00	0.02	1.60	3.08	4.26	0	0	0	0

close to 1, which indicates overestimation of makespan. This advantage remains for $p = 0.8$ and $\sigma \in \{0.1, 0.2, 0.3\}$. For most other scenarios, the six methods have similar performance. Furthermore, the results show how the reliability measure P_2 , or the service level, varies with uncertainty level σ and safety margin level p regarding task durations. Specifically, the service level increases as p increases when σ is fixed and decreases as σ increases when p is fixed. Given a desired service level and specified task uncertainty level, we can directly find the appropriate confidence level of task safety margins. For example, if we want a service level around 0.75, i.e., 75%, given $\sigma = 0.1$, our procedure achieves this goal with $p = 0.8$.

- For P_3 , Table 5 shows that procedure PP gives the smallest value for all scenarios of p and σ , indicating smaller feeding buffers than other methods. Similarly, the significant advantage of our method PP is observed at $p = 0.9$ and $\sigma = 0.5$, where our procedure delivers an average feeding buffer size of 2.44 and the C&PM, RSEM, APD and SMC procedures deliver much larger sizes of 9.55, 10.76, 14.30 and 18.13, respectively.
- For P_4 , the value of our procedure PP and Yu2013 is consistently zero for all p and σ , indicating no issue of a challenged critical chain. By contrast, the other benchmarks fail to avoid a challenged critical chain in most scenarios, as indicated by values of one.

To summarize, our procedure provides substantial improvements in P_1 , P_3 and P_4 over the benchmark methods for Project C201211. Because the decomposition of project networks potentially leads to shorter feeding chains and smaller limits of feeding buffers, our procedure directly leads to improvements in P_3 and P_4 . Based on detailed analysis of the network structure and relations between the buffers, the defined project buffer further integrates the safety margins of all minimal blocks of the project network, which include the safety margins of the critical tasks and the remaining safety margins of feeding chains. As a result, our procedure finds more accurate estimated project makespan values, as indicated by P_1 . As for P_2 , given that a reasonable value should be

between 0.6 and 0.9, the proposed procedure outperforms others for relatively high levels of p , such as 0.8 and 0.9, which fall in a range that is representative of practice.

6.2. RanGen projects

Sections 6.2.1 and 6.2.2 provide decomposition and buffer sizing results, respectively, for projects with data randomly generated by RanGen2.

6.2.1. Decomposition

Table 6 summarizes the decomposition results for each set of project networks. For each set, the average number of minimal blocks and the average number of tasks per minimal block are calculated respectively by

$$\sum_{\text{Network } i} \frac{\text{NoMB}(i)}{\text{NoN}}, \text{ and } \sum_{\text{Network } i} \frac{\text{NoT}}{\text{NoN} \cdot \text{NoMB}(i)},$$

where NoMB(i) is the number of minimal blocks for network i , NoN (= 30) is the number of networks within each set, and NoT (= 150) is the number of tasks in each network. Dummy tasks are not included in this calculation. As I_2 increases, indicating that the project network becomes closer to serial, we find more minimal blocks and the decomposition procedure works more effectively. Correspondingly, the average number of tasks per minimal block decreases, which enables efficient identification of feeding chains and calculation of feeding buffers within each minimal block.

6.2.2. Buffer sizing

Tables 7–9 summarize the results of our procedure and the five benchmarks on 90 randomly generated project networks. The indicators P'_1 , P'_2 and P'_3 are calculated as the average values of P_1 , P_2 and P_3 , respectively, over 30 project networks within each subset $I_2 \in \{0.25, 0.5, 0.75\}$ that have the same serial/parallel indicator. Also, P'_4 denotes the number of project networks with $P_4 = 1$ in

Table 8Comparative performance on projects with $I_2 = 0.5$.

$I_2 = 0.5$		P'_1 (MAD): 100%				P'_2 (REL): [0.1]				P'_3 (AFB)				P'_4 (No. of Projects with IND=1)			
		$p=0.6$	$p=0.7$	$p=0.8$	$p=0.9$	$p=0.6$	$p=0.7$	$p=0.8$	$p=0.9$	$p=0.6$	$p=0.7$	$p=0.8$	$p=0.9$	$p=0.6$	$p=0.7$	$p=0.8$	$p=0.9$
$\sigma=0.1$	PP	0.01	0.01	0.01	0.02	0.45	0.57	0.70	0.84	0.09	0.22	0.38	0.59	0	0	0	0
	C&PM	0.01	0.02	0.04	0.08	0.72	0.97	1.00	1.00	0.16	0.39	0.66	1.05	29	30	30	30
	RSEM	0.01	0.01	0.02	0.03	0.49	0.67	0.86	0.98	0.18	0.41	0.70	1.12	29	29	30	30
	APD	0.02	0.02	0.02	0.02	0.93	0.93	0.93	0.93	1.42	1.42	1.42	1.42	30	30	30	30
	SMC	0.04	0.07	0.13	0.23	0.96	1.00	1.00	1.00	1.91	2.95	4.18	5.88	30	30	30	30
$\sigma=0.2$	Yu2013	0.01	0.02	0.03	0.05	0.59	0.84	0.97	1.00	0.14	0.34	0.56	0.87	0	0	0	0
	PP	0.02	0.02	0.02	0.03	0.35	0.46	0.61	0.79	0.14	0.40	0.71	1.11	0	0	0	0
	C&PM	0.02	0.04	0.09	0.17	0.56	0.94	1.00	1.00	0.25	0.71	1.28	2.13	29	30	30	30
	RSEM	0.02	0.02	0.03	0.08	0.38	0.59	0.87	1.00	0.27	0.76	1.36	2.27	29	30	30	30
	APD	0.08	0.08	0.08	0.08	0.99	0.99	0.99	0.99	2.85	2.85	2.85	2.85	30	30	30	30
$\sigma=0.3$	SMC	0.19	0.31	0.48	0.73	1.00	1.00	1.00	1.00	5.12	7.19	9.65	13.16	30	30	30	30
	Yu2013	0.02	0.03	0.05	0.11	0.45	0.76	0.97	1.00	0.22	0.60	1.03	1.59	0	0	0	0
	PP	0.03	0.03	0.03	0.04	0.27	0.37	0.52	0.73	0.14	0.54	0.98	1.58	0	0	0	0
	C&PM	0.03	0.05	0.13	0.28	0.40	0.88	1.00	1.00	0.25	0.95	1.85	3.23	29	30	30	30
	RSEM	0.03	0.03	0.05	0.13	0.29	0.51	0.87	1.00	0.27	1.01	1.97	3.45	29	30	30	30
$\sigma=0.4$	APD	0.15	0.15	0.15	0.15	1.00	1.00	1.00	1.00	4.34	4.34	4.34	4.34	30	30	30	30
	SMC	0.46	0.68	0.95	1.34	1.00	1.00	1.00	1.00	9.48	12.65	16.43	21.83	30	30	30	30
	Yu2013	0.03	0.03	0.08	0.19	0.33	0.67	0.97	1.00	0.22	0.79	1.41	2.18	0	0	0	0
	PP	0.05	0.04	0.04	0.05	0.20	0.29	0.43	0.67	0.10	0.62	1.21	1.98	0	0	0	0
	C&PM	0.05	0.05	0.16	0.39	0.26	0.78	1.00	1.00	0.17	1.11	2.34	4.33	29	30	30	30
$\sigma=0.5$	RSEM	0.05	0.04	0.06	0.20	0.21	0.41	0.85	1.00	0.18	1.18	2.49	4.61	29	30	30	30
	APD	0.24	0.24	0.24	0.24	1.00	1.00	1.00	1.00	5.89	5.89	5.89	5.89	30	30	30	30
	SMC	0.81	1.11	1.48	2.03	1.00	1.00	1.00	1.00	14.55	18.95	24.23	31.79	30	30	30	30
	Yu2013	0.05	0.04	0.10	0.28	0.23	0.56	0.96	1.00	0.15	0.91	1.71	2.64	0	0	0	0
	PP	0.07	0.06	0.05	0.05	0.15	0.23	0.36	0.60	0.01	0.66	1.38	2.32	0	0	0	0
$\sigma=0.5$	C&PM	0.07	0.06	0.18	0.49	0.16	0.53	1.00	1.00	0.01	1.18	2.75	5.40	29	30	30	30
	RSEM	0.07	0.05	0.07	0.26	0.15	0.33	0.81	1.00	0.01	1.25	2.93	5.75	29	30	30	30
	APD	0.34	0.34	0.34	0.34	1.00	1.00	1.00	1.00	7.53	7.53	7.53	7.53	30	30	30	30
	SMC	1.21	1.60	2.09	2.81	1.00	1.00	1.00	1.00	20.67	26.40	33.30	43.34	30	30	30	30
	Yu2013	0.07	0.05	0.11	0.37	0.16	0.44	0.94	1.00	0.01	0.96	1.94	3.01	0	0	0	0

Table 9Comparative performance on projects with $I_2 = 0.75$.

$I_2 = 0.75$		P'_1 (MAD): 100%				P'_2 (REL): [0.1]				P'_3 (AFB)				P'_4 (No. of Projects with IND=1)			
		$p=0.6$	$p=0.7$	$p=0.8$	$p=0.9$	$p=0.6$	$p=0.7$	$p=0.8$	$p=0.9$	$p=0.6$	$p=0.7$	$p=0.8$	$p=0.9$	$p=0.6$	$p=0.7$	$p=0.8$	$p=0.9$
$\sigma=0.1$	PP	0.01	0.01	0.01	0.01	0.53	0.64	0.76	0.88	0.08	0.20	0.34	0.53	0	0	0	0
	C&PM	0.01	0.02	0.04	0.07	0.80	0.99	1.00	1.00	0.11	0.26	0.44	0.70	25	25	25	28
	RSEM	0.01	0.01	0.01	0.02	0.54	0.67	0.80	0.93	0.13	0.30	0.50	0.80	25	25	25	30
	APD	0.01	0.01	0.01	0.01	0.61	0.62	0.61	0.61	0.83	0.83	0.83	0.83	29	29	29	29
	SMC	0.02	0.04	0.07	0.14	0.86	0.99	1.00	1.00	1.61	2.91	4.43	6.53	29	30	30	30
$\sigma=0.2$	Yu2013	0.01	0.01	0.01	0.02	0.56	0.71	0.85	0.96	0.11	0.26	0.44	0.68	0	0	0	0
	PP	0.02	0.02	0.02	0.03	0.46	0.57	0.71	0.86	0.13	0.36	0.64	1.01	0	0	0	0
	C&PM	0.02	0.04	0.08	0.14	0.69	0.97	1.00	1.00	0.17	0.47	0.85	1.43	25	25	29	30
	RSEM	0.02	0.02	0.02	0.04	0.47	0.61	0.79	0.96	0.19	0.54	0.98	1.63	25	25	30	30
	APD	0.02	0.02	0.02	0.02	0.71	0.70	0.71	0.71	1.68	1.68	1.68	1.68	30	30	30	30
$\sigma=0.3$	SMC	0.06	0.14	0.27	0.47	0.98	1.00	1.00	1.00	3.88	6.46	9.49	13.82	30	30	30	30
	Yu2013	0.02	0.02	0.02	0.04	0.49	0.64	0.83	0.97	0.17	0.47	0.82	1.29	0	0	0	0
	PP	0.03	0.03	0.03	0.04	0.40	0.52	0.66	0.84	0.13	0.48	0.89	1.44	0	0	0	0
	C&PM	0.03	0.05	0.11	0.21	0.56	0.94	1.00	1.00	0.17	0.64	1.23	2.16	25	27	30	30
	RSEM	0.03	0.03	0.03	0.06	0.41	0.55	0.77	0.97	0.19	0.73	1.41	2.47	25	28	30	30
$\sigma=0.4$	APD	0.04	0.04	0.04	0.04	0.78	0.78	0.78	0.78	2.54	2.54	2.54	2.54	30	30	30	30
	SMC	0.16	0.32	0.55	0.90	1.00	1.00	1.00	1.00	6.79	10.74	15.47	22.20	30	30	30	30
	Yu2013	0.03	0.03	0.04	0.07	0.42	0.59	0.81	0.98	0.17	0.63	1.14	1.82	0	0	0	0
	PP	0.04	0.03	0.04	0.05	0.35	0.46	0.61	0.81	0.09	0.56	1.09	1.82	0	0	0	0
	C&PM	0.03	0.06	0.14	0.29	0.43	0.89	1.00	1.00	0.12	0.74	1.56	2.89	25	29	30	30
$\sigma=0.5$	RSEM	0.04	0.03	0.04	0.09	0.35	0.50	0.75	0.98	0.13	0.85	1.79	3.31	25	30	30	30
	APD	0.06	0.06	0.06	0.06	0.83	0.84	0.84	0.83	3.45	3.45	3.45	3.45	30	30	30	30
	SMC	0.31	0.57	0.89	1.35	1.00	1.00	1.00	1.00	10.44	15.79	22.15	31.26	30	30	30	30
	Yu2013	0.04	0.03	0.05	0.10	0.36	0.53	0.78	0.98	0.12	0.72	1.40	2.26	0	0	0	0
	PP	0.05	0.04	0.04	0.06	0.31	0.41	0.56	0.78	0.01	0.59	1.26	2.15	0	0	0	0
$\sigma=0.5$	C&PM	0.05	0.06	0.16	0.37	0.31	0.80	1.00	1.00	0.01	0.79	1.84	3.61	25	29	30	30
	RSEM	0.05	0.04	0.05	0.12	0.31	0.44	0.71	0.98	0.01	0.90	2.10	4.12	25	30	30	30
	APD	0.08	0.08	0.08	0.08	0.87	0.87	0.88	0.88	4.42	4.42	4.42	4.42	30	30	30	30
	SMC	0.51	0.84	1.26	1.87	1.00	1.00	1.00	1.00	14.64	21.52	29.81	41.66	30	30	30	30
	Yu2013	0.05	0.04	0.06	0.13	0.31	0.46	0.74	0.99	0.01	0.76	1.60	2.63	0	0	0	0

each subset. We note that since the APD method does not consider the confidence level p , its results for buffer size and performance indicators do not vary with p values when σ is fixed within each subset.

In general, the results of this computational study are consistent with those in the case study in Section 6.1, and demonstrate that our buffer sizing procedure has superior performance over the selected benchmarks regarding the four indicators. As indicated by smaller P'_1 values in Tables 7–9, procedure PP delivers more accurate project makespan estimates than the benchmarks for most scenarios of p and σ . This advantage is sustained as the serial/parallel network measure I_2 increases from 0.25 to 0.75. Typically, the P'_1 value of our method stays around 0.05, whereas the other methods deliver several times larger values for high levels of p and σ . For example, the SMC gives a value of 2.81 in the sce-

nario where $p = 0.9$ and $\sigma = 0.5$ for project networks with $I_2 = 0.5$ (Table 8).

For the reliability measure P'_2 , reasonable values should be between 0.6 and 0.9, as discussed in Section 5.1. We observe that procedure PP has superior performance at $p = 0.9$. More specifically, for project networks with medium-to-high serial/parallel measures, i.e. $I_2 \in \{0.5, 0.75\}$, the proposed procedure consistently provides reliability values within the range of [0.6, 0.9], whereas other methods produce very high values that are close to 1.0, indicating overestimation of project makespan. For most other scenarios of p and σ , these methods show comparable performance.

The smallest P'_3 value provided by our procedure PP shows that it delivers the smallest feeding buffers among all six methods, for all scenarios of I_2 , p and σ . Consider a representative scenario of $p = 0.8$ and $\sigma = 0.3$ for projects with $I_2 = 0.5$. Our procedure de-

Table 10Regression coefficients for n , and p -values.

	C&PM	RSEM	APD
RA(P_1)	-0.0035, $4.09e^{-20}$	-0.0026, $2.34e^{-10}$	-0.0054, $5.13e^{-12}$
RA(P_3)	-0.0027, $5.62e^{-7}$	-0.0017, $2.11e^{-6}$	-0.0010, $3.51e^{-6}$
	SMC	Yu2013	
RA(P_1)	-0.0025, $2.42e^{-8}$	-0.0038, $3.23e^{-14}$	
RA(P_3)	-9.57e ⁻⁴ , $1.95e^{-11}$	-0.0012, -0.0010	

Table 11One-way ANOVA p -values.

	C&PM	RSEM	APD	SMC	Yu2013
RA(P_1)	$2.28e^{-30}$	$1.30e^{-10}$	$4.11e^{-15}$	$1.58e^{-13}$	$2.50e^{-22}$
RA(P_3)	$1.38e^{-12}$	$2.00e^{-10}$	$4.75e^{-11}$	$6.67e^{-20}$	$5.36e^{-6}$

livers an average feeding buffer size of 0.98, whereas the others produce buffers with average size ranging from 1.41 to 16.43.

The results for P_4 again show that procedure PP and Yu2013 both avoid the issue of a challenged critical chain for all 90 instances, whereas the other benchmark methods do not.

For our procedure, the reliability measure P_2' decreases as σ increases given fixed p , and decreases as p decreases given fixed σ . Similar to our analysis in the case study, the results give information on how the reliability measure, or the service level, varies with levels of uncertainties and safety margins regarding task durations. Given a desired service level and a specified uncertainty level, we can find the task safety margins that imply a chosen confidence level.

We perform an additional experiment to study the sensitivity to project size of the relative advantage of our procedure over the five benchmarks as measured by P_1 and P_3 . We let $n \in \{30, 60, 90, 120, 150\}$ and generate 10 networks with $l_2 = 0.5$ for each n . We set $p = 0.8$, $\sigma = 0.3$ and simulate 1000 times for each network. The relative advantage (RA) of our procedure over each benchmark i is calculated as follows:

$$RA_i(P_1) = \frac{P_1(PP) - P_1(i)}{P_1(i)}, \quad RA_i(P_3) = \frac{P_3(PP) - P_3(i)}{P_3(i)}.$$

For this experiment, the smaller the negative RA value, the greater the advantage of our procedure. For each $RA_i(P_1)$ and $RA_i(P_3)$, regression analysis produces negative coefficients on n with p -values very close to zero, and one-way ANOVA also produces p -values very close to zero, as shown in Tables 10 and 11. These results indicate that the relative advantage of our procedure increases with the project size n .

The 50 networks generated in the sensitivity analysis are further used to test the performance of the buffer sizing methods considering different σ values for tasks within each project, for which we randomly assign a σ value among $\{0.1, 0.2, 0.3, 0.4, 0.5\}$ to every task. Also, we set $p = 0.8$ and simulate 1000 times for each network. The results regarding four performance indicators as well as sensitivity analysis are provided in Appendix H, showing similar advantages of the proposed method against the five benchmarks as discussed above.

In summary, the proposed procedure shows substantial improvement in terms of P_1 , P_3 and P_4 . It delivers more accurate estimated project makespans with smaller feeding buffers, and avoids the issue of a challenged critical chain. For the reliability measure P_2 , our procedure generally performs well for $p = 0.9$. This indicates that our procedure has the best performance when safe task durations are estimated at the 90%-confidence level.

7. Concluding remarks

Because of the large economic value at stake, the sizing of buffers is centrally important to CCPM. Despite extensive research, previous approaches suffer from two significant deficiencies, which occur at the planning stage of projects: erroneous buffer sizing leads to inaccurate estimation of project makespan, and the insertion of feeding buffers overrides the critical chain. To resolve these issues, we develop a buffer sizing procedure that decomposes the project network to obtain more accurate information about the relative lengths of critical and noncritical chains, and about interactions between buffers, over complex network structures. Hence, the size of a feeding buffer is determined based on uncertainties of all associated noncritical chains and their relations with the critical chain, instead of only the uncertainty of the longest chain feeding it. Moreover, feeding buffers are sized to keep the critical chain unchallenged, and the project buffer additionally incorporates remaining safety margins, by comparing feeding chains with their parallel critical counterparts. Based on extensive computational testing for both real and simulated data, our procedure provides more accurate buffer sizing and project makespan estimation, as well as smaller feeding buffers, than five widely used benchmark methods. Additional benefits include delayed expenditure, and reduced work-in-process, rework, and multitasking.

Our work should be of direct value to project management companies, for several reasons. First, all the information required for the network decomposition is immediately available in every well documented project under our problem setting. Second, the algorithmic steps required can easily be implemented as an add-on to commercial project planning software, such as Microsoft Project. Third, the elimination of the issue of challenging the critical chain simplifies project planning and reduces replanning. Fourth, the flexibility in using safety margins enables a project company to adjust its service level, in order to take into account strategic issues that frequently influence project selection and prioritization. Fifth, our buffer sizing procedure enables significantly more accurate and robust estimation of project makespan than earlier methods, thereby helping project companies to avoid the problems of underestimation and overestimation, and their significant costs or opportunity costs. Sixth, due to the enhancements of CCPM provided by our procedure, the choice that project companies face between using the traditional and CCPM planning may be clarified. Overall, we hope that the following comment will be helpful to project companies: our work enables significant reduction in buffers relative to other methods, but because the buffers we design are more accurately sized, the result is an improvement in project estimation.

Several problems remain open for future research. First, since our buffer sizing procedure depends on project network structure, it would be valuable to test it on project management applications with particular structures; an example would be assembly-type projects with many concurrent tasks early in the project but fewer later. Second, a widely used response to routine delays in project execution is fast tracking, meaning the concurrent processing of tasks that have a formal precedence relationship. It would be valuable to study how the need to allow for this possibility during execution can be incorporated into planning stage buffer-sizing decisions using network decomposition. Third, the more accurate planning of buffers that our procedure enables allows for the possibility of controllably running more tasks concurrently, which may improve the effectiveness of resource allocations for project companies. Finally, since many companies share their resources across concurrent projects, we recommend extension of our buffer sizing procedure to the planning of multiple projects that share resources.

CRedit authorship contribution statement

Bingling She: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Resources, Data curation, Writing - original draft, Writing - review & editing, Visualization, Funding acquisition. **Bo Chen:** Conceptualization, Methodology, Validation, Formal analysis, Investigation, Resources, Data curation, Writing - review & editing, Visualization, Supervision, Project administration, Funding acquisition. **Nicholas G. Hall:** Conceptualization, Methodology, Validation, Formal analysis, Investigation, Resources, Data curation, Writing - review & editing, Visualization, Funding acquisition.

Acknowledgements

This research is supported in part by the China Scholarship Council, to the first author; and in part by the [National Natural Science Foundation of China](#) under Grant 71732003 and by the Berry Professorship at the Fisher College of Business, The Ohio State University, to the third author.

Appendix A. Acronyms

We summarize in [Table A.1](#) the acronyms used in the paper.

Table A.1
Abbreviations.

Acronym	Explanation
BSM	Block Safety Margin
BTI	Block Time Interval
CCP	Critical Counterpart
CCPM	Critical Chain Project Management
C&PM	Cut-and-Paste Method
CTI	Critical Time Interval
EST	Earliest Start Time
EFC	Effective Feeding Chain
EFT	Earliest Finish Time
FB	Feeding Buffer
FC	Feeding Chain
FCSM	Feeding Chain Safety Margin
FT	Finish Time (Critical Task)
LFT	Latest Finish Time
LST	Latest Start Time
PB	Project Buffer
PN	Project Network
RCPSP	Resource Constrained Project Scheduling Problem
RSEM	Root Square Error Method
RSM	Remaining Safety Margin
SM	Safety Margin
ST	Start Time (Critical Task)

Appendix B. The model of RCPSP

In the following model, s_i is the decision variable for the start time of each task; $T_{\max} = \sum_{v_i \in V} d_i$ denotes an upper bound on the duration of the critical chain, since a project can be scheduled within the time period of $[0, T_{\max}]$ [19]; and the binary variable x_{it} indicates whether or not task v_i is active in time period t .

$$\begin{aligned}
 & \text{(RCPSP)} \quad \min \quad T \\
 & \text{s.t.} \quad s_i + d_i \leq T, \quad \forall v_i \in V, \\
 & \quad s_i + d_i \leq s_j, \quad \forall (v_i, v_j) \in E, \\
 & \quad \sum_{v_i \in V} r_{ik} \cdot x_{it} \leq r_k, \quad \forall k \in R, \\
 & \quad \forall t \in \{1, \dots, T_{\max} + 1\}, \\
 & \quad s_i \geq 0, \quad \forall v_i \in V, \\
 & \quad x_{it} = \begin{cases} 1, & \text{if } s_i \leq t - 1 < s_i + d_i, \\ 0, & \text{otherwise.} \end{cases} \\
 & \quad \forall v_i \in V, \quad \forall t \in \{1, \dots, T_{\max} + 1\}
 \end{aligned}$$

Appendix C. Heuristic to solve RCPSP

The heuristic starts from a critical-path-based schedule, and focuses on breaking the resource contentions by introducing precedence relations between tasks. Given an acyclic task-on-node network $PN(V, E)$, for any task $i \in V$, let $IP(i)$ and $IS(i)$ denote the set of its immediate predecessors and the set of its immediate successors, respectively, and let $EST(i)$, $LST(i)$ and $TF(i)$ denote its earliest start time, latest start time and total float (or total slack), respectively.

- Step 1. For every task $i \in V$, based on E compute $IP(i)$, $IS(i)$, $EST(i)$, $LST(i)$ and $TF(i)$. Let $t = LST(n)$.
- Step 2. [Identify the last resource conflict in the network of late finish times.] Compute the active set $AS(t)$ in time period t :

$$AS(t) = \{i \in V : LST(i) \leq t - 1 < LST(i) + d(i)\},$$
 and then the resource residuals $R_k = \sum_{i \in AS} r_{ik} - r_k$, for $k \in R$. If $R_k \leq 0$ for all $k \in R$, then set $t := t - 1$; otherwise, go to Step 3. If $t > 0$, then return to the start of Step 2; otherwise, stop.
- Step 3. [Find the resource conflict sets in time period t .] For any $k \in R$ with $R_k > 0$, compute $RCS_k(t) = \{i \in AS(t) : r_{ik} > 0\}$, and calculate l_k , the number of tasks with zero total float in $RCS_k(t)$.
- Step 4. [Deal with resource conflict sets $RCS_k(t)$ in ascending order of l_k .] For all relevant $k \in R$ identified in Step 3, add a new precedence relation between two specified tasks in $RCS_k(t)$: find the tasks $i, j \in RCS_k(t)$ with the smallest $EST(i)$ and the largest $LST(j)$, $i \neq j$, set $E := E \cup \{(i, j)\}$, $IS(i) := IS(i) \cup \{j\}$, $IP(j) := IP(j) \cup \{i\}$. For tasks with the same EST or LST , choose the task with smaller duration. Go to Step 1.

We use the critical path procedure on the extended project network derived from the arc set E output by the above heuristic to identify the critical chain and the baseline schedule of late start times.

Appendix D. Data of extended project network C201211

Table D.1
Data of extended network of project C201211.

Task label	Mean duration d	Immediate successors	Safety margin ($\sigma = 0.3, p = 0.8$)
1	0	2,3,4	0
2	12	5,6	2.77
3	11	5,6	2.54
4	6	5,6	1.38
5	17	7,8	3.92
6	14	7,8	3.23
7	6	9,10,11	1.38
8	10	9,10,11	2.31
9	4	12,13	0.92
10	2	12,13	0.46
11	7	12	1.61
12	6	14	1.38
13	5	14	1.15
14	0		0

Appendix E. Time complexity of algorithm MB

Proposition 2. Algorithm MB finds the minimal blocks of network $PN(V, \tilde{E})$ in $O(n^2)$ time.

Proof. Given a critical chain, Step (1) identifies the smallest possible initial time intervals. Steps (1)–(3) ensure that the conditions of a block are satisfied and the generated BTIs cannot be divided without violating these conditions. Hence, all minimal blocks are found by construction. Regarding the time complexity, the Initiation step finds a longest path in an acyclic network, which requires $O(n^2)$ time. Step (1) requires $O(n)$ time for each task. Steps (2) and (3) merge $O(n)$ time intervals. Each such merger reduces the number of remaining time intervals by one, and requires $O(n)$ time to merge the related tasks. Hence, these steps require $O(n^2)$ time. Therefore, the overall time requirement of the algorithm is $O(n^2)$. $\square$

Appendix F. Time complexity of algorithm BSM

Proposition 3. The time complexity of Algorithm BSM is $O(\#(B))$, where $\#(B)$ denotes the number of effective feeding chains in block B .

Proof. The main operation in the procedure is to calculate the maximum values as described in Steps (1)–(3), and the number of $\max\{\cdot\}$ operations is equal to the number of effective feeding chains. $\square$

Appendix G. Time complexity of the buffer sizing procedure

Proposition 4. For a project $PN(V, E)$, the time complexity of our buffer sizing procedure is $O(T_{\max}n^2 \log n + \#(PN) + \mathcal{L}(\#(PN), n))$.

Proof. Let $T_{\max} = \sum_{i=1}^n d_i$. In CCPM, since each task requires one resource in R , we have $m \leq n$. Hence, the time required by Heu0 to solve RCPSP, as described in Appendix C, is $O(T_{\max}n^2 \log n)$. From Proposition 2, the time required for the block decomposition procedure is $O(n^2)$. The resulting number of minimal blocks is $O(n)$. Within the project, the number of precedence relationships is $O(n^2)$, and the number of feeding buffers is $O(n)$. Given a critical chain, the number of effective feeding chains is

$O(\#(PN))$, which is the sum of $\#(B)$ s of all blocks B . For all minimal blocks, the time required for sizing the feeding buffers is $O(n^2 + \#(PN) + \mathcal{L}(\#(PN), n))$. This includes the time of depth-first search on auxiliary graphs, finding and calculating feeding chain safety margins, and solving the linear programs to calculate upper limits on feeding buffers. The time required for calculating block safety margins is $O(\#(PN))$. Finally, the computation of the project buffer in (6) requires $O(n)$ time. Therefore, the overall time requirement of our buffer sizing procedure is $O(T_{\max}n^2 \log n + \#(PN) + \mathcal{L}(\#(PN), n))$. $\square$

Appendix H. Results of an experiment on varied standard deviation

Table H.1 summarises the average performance measures for each project size $n = 30, 60, 90, 120$ and 150 . Tables H.2 and H.3

Table H.2

Varied σ : regression coefficients for n , and p -values.

	C&PM	RSEM	APD
RA(P_1)	−0.0037, $1.06e^{-19}$	−0.0024, $5.64e^{-9}$	−0.0057, $8.18e^{-13}$
RA(P_3)	−0.0027, $1.22e^{-6}$	−0.0017, $5.28e^{-6}$	−0.0010, $4.82e^{-6}$
	SMC	Yu2013	
RA(P_1)	−0.0027, $3.13e^{-8}$	−0.0037, $1.21e^{-13}$	
RA(P_3)	−8.75 e^{-4} , $1.43e^{-9}$	−0.0011, −0.0032	

Table H.3

Varied σ : one-way ANOVA p -values.

	C&PM	RSEM	APD	SMC	Yu2013
RA(P_1)	$5.58e^{-27}$	$3.37e^{-8}$	$1.24e^{-17}$	$1.86e^{-14}$	$2.15e^{-18}$
RA(P_3)	$3.09e^{-10}$	$2.31e^{-8}$	$1.43e^{-8}$	$8.46e^{-15}$	$1.23e^{-4}$

provide the sensitivity analysis results regarding regression coefficients (with p -values) and One-way ANOVA p -values, respectively.

Table H.1

Varied σ : comparative performance measures.

n	Methods	P'_1 (MAD): 100%	P'_2 (REL): [0,1]	P'_3 (AFB)	P'_4 (No. of Projects with IND=1)
30	PP	0.09	0.67	0.99	0
	C&PM	0.12	0.84	1.28	8
	RSEM	0.10	0.73	1.55	8
	APD	0.11	0.75	3.46	10
	SMC	0.24	0.97	6.14	10
	Yu2013	0.11	0.78	1.27	0
60	PP	0.06	0.54	0.83	0
	C&PM	0.10	0.91	1.17	10
	RSEM	0.07	0.71	1.50	10
	APD	0.11	0.90	3.39	10
	SMC	0.71	1.00	11.04	10
	Yu2013	0.08	0.78	1.14	0
90	PP	0.05	0.54	0.86	0
	C&PM	0.13	0.98	2.15	10
	RSEM	0.07	0.83	2.22	10
	APD	0.26	1.00	6.03	10
	SMC	0.86	1.00	14.49	10
	Yu2013	0.11	0.95	1.52	0
120	PP	0.04	0.49	0.93	0
	C&PM	0.13	1.00	2.06	10
	RSEM	0.06	0.84	2.13	10
	APD	0.23	1.00	5.50	10
	SMC	1.05	1.00	17.51	10
	Yu2013	0.09	0.96	1.37	0
150	PP	0.04	0.50	0.94	0
	C&PM	0.13	1.00	1.87	10
	RSEM	0.06	0.85	2.09	10
	APD	0.22	0.99	5.19	10
	SMC	1.13	1.00	18.68	10
	Yu2013	0.08	0.95	1.48	0

References

- [1] Ashtiani B, Jalali GR, Aryanezhad MB, Makui A. A new approach for buffer sizing in critical chain scheduling. In: IEEE international conference on industrial engineering and engineering management; 2007. p. 1037–41.
- [2] Batselier J, Vanhoucke M. Construction and evaluation framework for a real-life project database. *Int J Proj Manag* 2015;33(3):697–710.
- [3] Bernstein D, Rodeh M, Gertner I. On the complexity of scheduling problems for parallel/pipelined machines. *IEEE Trans Comput* 1989;38(9):1308–13.
- [4] Bie L, Cui N, Zhang X. Buffer sizing approach with dependence assumption between activities in critical chain scheduling. *Int J Prod Res* 2012;50(24):7343–56.
- [5] Cormen TH, Leiserson CE, Rivest RL. Introduction to algorithms. 3rd ed. MIT Press; 2009.
- [6] Demeulemeester E, Herroelen W. Project scheduling: a research handbook. Kluwer Academic Publishers; 2002.
- [7] Demeulemeester E, Vanhoucke M, Herroelen W. RanGen: a random network generator for activity-on-the-node networks. *J Sched* 2003;6:17–38.
- [8] Fallah M, Ashtiani B, Aryanezhad MB. Critical chain project scheduling: Utilizing uncertainty for buffer sizing. *Int J Res Rev Appl Sci* 2010;3(3):280–9.
- [9] Gao P, Feng J, Wang H. Grey critical chain project scheduling and its application. *Can Soc Sci* 2007;3(3):35–42.
- [10] Ghoddousi P, Ansari R, Makui A. A risk-oriented buffer allocation model based on critical chain project management. *KSCE J Civ Eng* 2017;21(5):1536–48.
- [11] Goldratt EM. Critical chain. USA: The North River Press; 1997.
- [12] Hall NG. Research and teaching opportunities in project management. In: Tutorials in operations research: optimization challenges in complex, networked and risky systems. INFORMS; 2016. p. 329–88. <https://doi.org/10.1287/educ.2016.0146>
- [13] Herroelen W, Leus R. On the merits and pitfalls of critical chain scheduling. *J Oper Manag* 2001;19:559–77.
- [14] Herroelen W, Leus R, Demeulemeester E. Critical chain project scheduling - do not oversimplify. *Proj Manag J* 2002;33(4):46–60.
- [15] Hoel K, Taylor SG. Quantifying buffers for project schedules. *Prod Invent Manag J* 1999;40(2):43–7.
- [16] Hu X, Cui N, Demeulemeester E. Effective expediting to improve project due date and cost performance through buffer management. *Int J Prod Res* 2015;53(5):1460–71.
- [17] Kokoskie G. A comparison of critical chain project management (CCPM) buffer sizing techniques. Fairfax: George Mason University; 2001.
- [18] Kolisch R, Padman R. An integrated survey of deterministic project scheduling. *Omega* 2001;29:249–72.
- [19] Lawler EL. Optimal sequencing of a single machine subject to precedence constraints. *Manag Sci* 1973;19(5):544–6.
- [20] Leach L. Schedule and cost buffer sizing: how to account for the bias between project performance and your model. *Proj Manag J* 2003;34(2):34–47.
- [21] Leach LP. Critical chain project management improves project performance. *Proj Manag* 1999;30(2):39–51.
- [22] Leach LP. Critical chain project management. 3rd ed. USA: The Artech House Publishers; 2014.
- [23] Long LD, Ohsato A. Fuzzy critical chain method for project scheduling under resource constraints and uncertainty. *Int J Proj Manag* 2008;26:688–98.
- [24] Ma G, Hao K, Xiao Y, Zhu T. Critical chain design structure matrix method for construction project scheduling under rework scenarios. *Math Probl Eng* 2019;1–14.
- [25] Ma G, Jiang Y. Improved fuzzy critical chain buffer setting method. *Int Conf Inf Manag Innov Manag Ind Eng (ICIII)* 2012;1:240–3.
- [26] Ma G, Wang A, Li N, Gu L. Improved critical chain project management framework for scheduling construction projects. *J Constr Eng Manag* 2014;140(12):1–12.
- [27] Newbold RC. Project management in the fast lane – applying the theory of constraints. USA: CRC Press; 1998.
- [28] OR&S. Operations Research and Scheduling Research Group - RanGen. <http://www.projectmanagement.ugent.be/research/data/RanGen>; 2019b. Last accessed 30 Nov 2020.
- [29] OR&S. Operations Research and Scheduling Research Group - Real data. <http://www.projectmanagement.ugent.be/research/data/realdata>; 2019a. Last accessed on 30 Nov 2020.
- [30] Patrick F. Critical chain scheduling and buffer management. Pennsylvania: Project Management Institute; 1998.
- [31] Peng W, Huang M. A critical chain project scheduling method based on a differential evolution algorithm. *Int J Prod Res* 2014;52(13):3940–9.
- [32] PMI. A guide to the project management body of knowledge. 5th ed. USA: Project Management Institute; 2013.
- [33] Raz T, Barnes R, Dvir D. A critical look at critical chain project management. *Proj Manag J* 2003;34(4):24–32.
- [34] Shi Q, Gong T. An improved project buffer sizing approach to critical chain management under resources constraints and fuzzy uncertainty. In: International conference on artificial intelligence and computational intelligence; 2009. p. 486–90.
- [35] Tenera AB. Critical chain buffer sizing: a comparative study. In: 2008 PMI research conference: defining the future of project management. Warsaw, Poland: PMI; 2008. p. 1–14.
- [36] Trietsch D. The effect of systemic errors on optimal project buffers. *Int J Proj Manag* 2005;23(4):267–74.
- [37] Trietsch D. Optimal feeding buffers for projects or batch supply chains by an exact generalization of the newsvendor result. *Int J Prod Res* 2006;44(4):627–37.
- [38] Tukul OI, Rom WO, Eksioglu SD. An investigation of buffer sizing techniques in critical chain scheduling. *Eur J Oper Res* 2006;172:401–16.
- [39] Vanhoucke M. Using activity sensitivity and network topology information to monitor project time performance. *Omega* 2010;38:359–70.
- [40] Vanhoucke M. On the dynamic use of project performance and schedule risk information during project tracking. *Omega* 2011;39:416–26.
- [41] Vanhoucke M. Project management with dynamic scheduling. 2nd ed. Springer; 2012.
- [42] Vanhoucke M, Coelho J, Batselier J. An overview of project data for integrated project management and control. *J Mod Proj Manag* 2016;3(3):6–21.
- [43] Vanhoucke M, Coelho J, Debels D, Maenhout B, Tavares L. An evaluation of the adequacy of project network generators with systematically sampled networks. *Eur J Oper Res* 2008;187(2):511–24.
- [44] Xie XM, Guang Y, Chuang L. Software development projects IRSE buffer settings and simulation based on critical chain. *J China Univ Posts Telecommun* 2010;17:100–6.
- [45] Yu J, Xu Z, Hu C. Buffer sizing approach in critical chain project management under multiresource constraints. In: Proceedings of 2013 6th international conference on information management, innovation management and industrial engineering (ICIII), 3. IEEE; 2013. p. 71–5.
- [46] Zarghami S, Gunawan I, Zubielqui G, Baroudi B. Incorporation of resource reliability into critical chain project management buffer sizing. *Int J Prod Res* 2019. <https://doi.org/10.1080/00207543.2019.1667041>
- [47] Zhang J, Li Y, Wang R. An assembly task scheduling method for aircraft development project based on critical chain. *Manag Serv Sci* 2009;2009:1–4.
- [48] Zhang J, Song X, Chen H, Shi R. Determination of critical chain project buffer based on information flow interactions. *J Oper Res Soc* 2016b;67:1146–57.
- [49] Zhang J, Song X, Díaz E. Buffer sizing of critical chain based on attribute optimization. *Concurr Eng* 2014;22(3):253–64.
- [50] Zhang J, Song X, Díaz E. Project buffer sizing of a critical chain based on comprehensive resource tightness. *Eur J Oper Res* 2016a;248(1):174–82.
- [51] Zhang J, Song X, Díaz E. Critical chain project buffer sizing based on resource constraints. *Int J Prod Res* 2017;55(3):671–83.
- [52] Zhao W, Hall NG, Liu Z. Project evaluation and selection with task failures. *Prod Oper Manag* 2020;29(2):428–46. <https://doi.org/10.1111/poms.13107>
- [53] Zhao Y, Cui N, Tian W. A two-stage approach for the critical chain project rescheduling. *Ann Oper Res* 2019:1–29.